

Sistema de transacciones financieras por voz en dispositivos móviles iOS

Ing. Alexis Geraldine Barniquez Piñero

Carrera de Especialización en Inteligencia Artificial

Director: Ing. Sergio Ivaldi

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, junio de 2026

Resumen

En la presente memoria se desarrolla una prueba de concepto de una aplicación móvil nativa para iOS, capaz de procesar comandos de voz e interpretar intenciones transaccionales de forma 100 % local. Este sistema fue desarrollado para ser validado por el equipo de Identidad Digital del Banco Galicia Minorista.

Para su materialización, se utilizaron herramientas del ecosistema Apple como Swift, Xcode, la API AVFoundation y el framework Core ML.

Índice general

Resumen	I
1. Introducción general	1
1.1. Contexto de la banca móvil	1
1.2. Estado del arte	2
1.3. Objetivos y alcance	3
1.3.1. Alcance y limitaciones	3
1.4. Motivación: accesibilidad e inclusión	4
2. Introducción específica	5
2.1. Marco metodológico	5
2.2. Herramientas utilizadas	5
2.3. Requerimientos y modelos de IA utilizados	6
2.4. Exploración teórica de arquitecturas evaluadas	7
3. Implementación y desarrollo	9
3.1. Arquitectura del sistema	9
3.2. Tratamiento y preparación de datos	13
3.2.1. Creación de plantillas de comandos y aumentación de datos	13
3.2.2. Modelo de datos	13
3.3. Desarrollo del SDK (Aurum)	14
3.3.1. Clase fachada: <code>AurumEngine</code>	15
3.3.2. Captura de audio: <code>AudioCaptureManager</code>	16
3.3.3. Reconocimiento de voz: <code>SpeechRecognizer</code>	17
3.3.4. Normalización de texto: <code>TextNormalizer</code>	18
3.3.5. Procesamiento NLU: <code>NLUProcessor</code>	19
Clasificación de intenciones	19
Extracción de entidades nombradas (NER)	20
3.3.6. Modelo de datos: <code>TransactionPayload</code>	21
3.3.7. API pública y comunicación asíncrona	22
3.3.8. Gestión de errores: <code>AurumError</code>	22
3.4. Desarrollo de la app base	23
3.4.1. Arquitectura MVVM y flujo de estados	24
3.4.2. Integración con el SDK Aurum	25
3.4.3. Capa de presentación (SwiftUI)	27
Vista principal (<code>MainView</code>)	27
Vista de confirmación (<code>ConfirmationView</code>)	28
3.4.4. Gestión de permisos y privacidad	28
Privacidad <i>on-device</i>	29
3.4.5. Flujo de interacción del usuario	30
4. Ensayos y resultados	35
4.1. Desempeño del modelo	35

4.1.1.	Clasificador de intenciones (MaxEnt)	35
4.1.2.	Extractor de entidades (CRF)	36
	Métricas a nivel de token	36
	Métricas a nivel de entidad	37
4.2.	Ensayo de modelos	38
4.2.1.	Evaluación de robustez lingüística	38
4.2.2.	Benchmark de latencia de inferencia	40
4.3.	Pruebas de usabilidad	41
4.3.1.	Protocolo de evaluación	41
4.3.2.	Resultados de usabilidad	42
4.4.	Análisis de resultados	43
4.4.1.	Validación de la hipótesis central	43
4.4.2.	Comparación con arquitecturas alternativas	43
4.4.3.	Rol del <code>TextNormalizer</code>	44
4.4.4.	Limitaciones del estudio	44
4.4.5.	Conexión con CRISP-DM	45
	Bibliografía	47

Índice de figuras

3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.	11
3.2. Flujo de pipeline de una transacción realizada por voz.	12
3.3. Diagrama de secuencia UML del flujo de interacción completo. Se muestra la comunicación entre <code>MainView</code> , <code>TransactionViewModel</code> , <code>AurumEngine</code> , <code>SpeechRecognizer</code> y <code>NLUProcessor</code>	32
3.4. Diagrama de flujo de Aurum.	33

Índice de tablas

2.1. Comparativa de arquitecturas evaluadas para NLU <i>on-device</i> en iOS. Los valores de BETO y BiLSTM corresponden a referencias bibliográficas; los de Create ML (MaxEnt/CRF) son resultados experimentales propios.	8
3.1. Componentes del SDK Aurum y sus dependencias de frameworks nativos.	15
3.2. Permisos requeridos por la app base y su justificación técnica. . . .	29
4.1. Rendimiento del clasificador de intenciones (MaxEnt) evaluado sobre el conjunto de test.	36
4.2. Matriz de confusión del clasificador de intenciones sobre el conjunto de test. Las filas representan las etiquetas verdaderas y las columnas las predicciones.	36
4.3. Rendimiento del extractor de entidades (MLWordTagger, algoritmo CRF) a nivel de token individual con esquema IOB.	37
4.4. Rendimiento del extractor de entidades a nivel de entidad completa. <i>Strict</i> : coincidencia exacta de tipo y posición. <i>Relaxed</i> : coincidencia de tipo con superposición parcial de tokens.	38
4.5. Evaluación de robustez del pipeline NLU ante variaciones lingüísticas del español rioplatense y errores de STT. <i>Intent Acc.</i> : accuracy de clasificación de intención. <i>Entity F1</i> : macro F1-Score <i>strict</i> de extracción de entidades.	39
4.6. Latencia de inferencia de los modelos Core ML, medida sobre macOS (CPU). En un dispositivo iOS con Neural Engine, las latencias serían inferiores. Se reportan estadísticos sobre 1.600 inferencias por modelo.	40
4.7. Resultados de las pruebas de usabilidad por tarea. La tasa de éxito se calcula sobre el total de intentos. El tiempo de interacción incluye el habla del usuario, el procesamiento y la confirmación.	42
4.8. Resumen de rendimiento de los modelos Core ML del SDK Aurum.	46

Capítulo 1

Introducción general

El presente capítulo enmarca el desarrollo de una solución basada en interfaces conversacionales impulsadas por inteligencia artificial. A lo largo de las siguientes secciones se analizará el estado del arte de estas tecnologías, justificando la necesidad crítica de implementar un procesamiento local de los datos para garantizar la seguridad de la información financiera, y se define de manera precisa los objetivos y el alcance metodológico de la prueba de concepto a desarrollar en el ecosistema iOS

1.1. Contexto de la banca móvil

En la última década, la banca digital ha experimentado una transformación radical [1], al migrar de plataformas web tradicionales a aplicaciones móviles que se han consolidado como el principal punto de contacto entre los clientes y las instituciones financieras. En este entorno de alta competitividad, la innovación continua en la experiencia de usuario se ha vuelto un diferenciador estratégico clave para la retención y satisfacción del cliente. Las interfaces conversacionales, impulsadas por los recientes avances en inteligencia artificial y procesamiento de lenguaje natural, representan la siguiente frontera en la interacción humano-computadora, lo que permite una comunicación más fluida, rápida e intuitiva [2]. Este trabajo se enmarca en dicha evolución tecnológica, explorando la viabilidad de aplicar tecnologías de reconocimiento y comprensión de voz para la ejecución de operaciones financieras directamente desde el ecosistema de una aplicación de banca móvil.

En el contexto argentino en particular, la adopción de la banca móvil se aceleró de manera significativa a partir de la pandemia de COVID-19, que en 2020 forzó a las instituciones financieras a consolidar sus canales digitales como alternativa al servicio presencial [3]. Según datos del Banco Central de la República Argentina (BCRA), el número de transferencias inmediatas a través de plataformas digitales creció sostenidamente en los años subsiguientes, y el segmento de clientes que gestiona sus finanzas exclusivamente a través del canal móvil representa hoy una fracción sustancial de la base de usuarios de los principales bancos privados del país [4]. El Banco Galicia [5], como una de las entidades financieras privadas de mayor envergadura del país, no es ajeno a esta tendencia: sus plataformas digitales concentran la gran mayoría de las interacciones cotidianas de sus clientes, como consultas de saldo, transferencias, pagos de servicios y operaciones de inversión.

Es en este escenario de alta digitalización y creciente demanda de experiencias de usuario personalizadas donde la interfaz de voz emerge como una propuesta de valor diferencial. La interacción por voz reduce la carga cognitiva asociada a la navegación entre menús y formularios, y permite ejecutar operaciones en contextos donde el uso de las manos no es posible o no resulta conveniente. Esta convergencia entre la madurez tecnológica de los modelos de lenguaje y la infraestructura de hardware de los dispositivos móviles modernos crea las condiciones necesarias para desarrollar soluciones de voz robustas, seguras y completamente autónomas que no dependan de servicios externos.

1.2. Estado del arte

En la actualidad, los asistentes de voz de propósito general han alcanzado un grado de madurez notable en tareas de reconocimiento automático de habla y comprensión del lenguaje natural [6, 7]. Sin embargo, la adopción e integración de estas tecnologías en dominios de misión crítica y alta seguridad, como el sector bancario, sigue siendo limitada. Las soluciones bancarias que han incorporado interfaces conversacionales suelen basarse en integraciones en la nube, donde los flujos de audio y datos se envían a servidores externos para su procesamiento. Este enfoque tradicional genera legítimas preocupaciones respecto a la privacidad, la latencia y la exposición de información financiera sensible [8].

Entre los casos de referencia más relevantes a nivel internacional, se destaca Erica [9], el asistente virtual de Bank of America, lanzado en 2018 y considerado uno de los primeros asistentes financieros conversacionales en alcanzar escala masiva. Erica permite a los usuarios realizar consultas de saldo, revisar el historial de transacciones y recibir recomendaciones personalizadas mediante lenguaje natural. Sin embargo, su arquitectura depende del procesamiento en la nube, lo que implica la transmisión continua de datos financieros sensibles a servidores externos. De manera similar, iniciativas como Eno de Capital One [10] y los chatbots bancarios desplegados en diversas entidades europeas siguen el mismo paradigma de conectividad remota.

En el ecosistema de Apple, la integración de asistentes de voz en aplicaciones financieras ha explorado el uso de SiriKit [11], un framework que permite a los desarrolladores exponer ciertas funcionalidades de sus aplicaciones al asistente nativo del sistema operativo. No obstante, esta integración presenta limitaciones en cuanto a la personalización del flujo conversacional y a la capacidad de interpretar intenciones transaccionales complejas y específicas del dominio bancario. El control sobre el procesamiento del lenguaje natural recae sobre los servidores de Apple, lo que no garantiza el nivel de privacidad requerido para operaciones financieras sensibles.

Para mitigar estos riesgos, la tendencia tecnológica emergente apunta hacia el procesamiento en el dispositivo. Bajo este paradigma, los modelos de inteligencia artificial se ejecutan localmente utilizando el hardware del dispositivo móvil, lo que garantiza que los datos acústicos y las intenciones transaccionales del usuario nunca abandonen su teléfono. Esta arquitectura maximiza la seguridad y la confidencialidad, pilares fundamentales para la viabilidad de cualquier solución dentro de la industria financiera. El avance en la comprensión de modelos de lenguaje, técnicas como la cuantización y la destilación de conocimiento, han hecho factible ejecutar modelos de comprensión del lenguaje natural de alta precisión

directamente sobre el procesador neuronal (*Neural Engine*) de los chips de la serie A y M de Apple [12, 13], lo que abrió una nueva generación de aplicaciones de IA privadas y sin latencia de red.

1.3. Objetivos y alcance

El objetivo principal de este trabajo es diseñar, desarrollar y evaluar un prototipo funcional de un asistente transaccional por voz integrado en el entorno de una aplicación móvil para iOS. Este prototipo servirá como una prueba de concepto destinada a validar la viabilidad técnica de un sistema que permita a los usuarios ejecutar operaciones financieras mediante comandos de voz en lenguaje natural.

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Mejorar la accesibilidad e inclusión: desarrollar un canal de interacción alternativo que empodere a personas con discapacidades visuales o motoras, que reduzca la dependencia de la navegación táctil tradicional.
- Garantizar la seguridad y privacidad de los datos: implementar modelos de machine learning optimizados para ejecutarse de manera estrictamente local mediante Core ML de Apple [14], asegurando que el audio y los datos sensibles del usuario se procesen íntegramente en el dispositivo.
- Optimizar la experiencia de usuario: simplificar operaciones bancarias complejas condensándolas en un único paso conversacional, y reduzca de esta manera la fricción y el tiempo necesario para completar una transacción.
- Sentar bases técnicas para futuras auditorías: generar la documentación arquitectónica y técnica necesaria para facilitar una futura evaluación y validación por parte del equipo de ciberseguridad de la entidad bancaria.

1.3.1. Alcance y limitaciones

Con el propósito de mantener el enfoque en la validación tecnológica del motor de inteligencia artificial y la experiencia de usuario, el trabajo se delimita bajo parámetros estrictos de alcance.

- Dentro del alcance: el desarrollo contempla exclusivamente la creación de una aplicación prototipo nativa para iOS, que operará de manera independiente y aislada. El núcleo técnico abarcará la implementación del pipeline de inteligencia artificial en el dispositivo físico, integrando la captura segura de audio, un modelo de reconocimiento automático de habla y un módulo de interpretación de lenguaje natural para extraer intenciones y entidades clave. Dado el carácter de la prueba de concepto, todo el flujo de transacción, la pantalla de confirmación y los resultados de las operaciones se simularán utilizando datos completamente ficticios.
- Fuera del alcance: el trabajo no contempla la integración del prototipo, ni de sus componentes, con el código fuente de la aplicación productiva oficial, ni la conexión con APIs, servicios de backend o bases de datos reales de la institución financiera. Asimismo, queda excluida la puesta en marcha en producción y la publicación en la App Store. Desde el punto de vista del manejo de la información, está estrictamente excluido el uso o acceso a

datos reales de clientes, así como la ejecución de transacciones que movi-
licen dinero real. Finalmente, el sistema se limitará al español rioplatense
focalizado, dejando fuera del alcance el soporte multilingüe y operaciones
financieras distintas a transferencias y consultas.

1.4. Motivación: accesibilidad e inclusión

El paradigma de diseño predominante en las aplicaciones de banca móvil actuales se basa casi exclusivamente en la interacción táctil y visual. Si bien este enfoque resulta eficiente para la mayoría de los usuarios, presenta barreras tecnológicas y de usabilidad significativas para grupos demográficos importantes. Por un lado, las personas con discapacidades o disminuciones visuales enfrentan el desafío de navegar a través de menús complejos, campos de texto y botones, tareas que a menudo resultan frustrantes incluso con el soporte de las herramientas de accesibilidad nativas de los sistemas operativos. Por otro lado, las personas con discapacidades motoras encuentran que la necesidad de una manipulación precisa de la pantalla dificulta o impide directamente el uso autónomo de la aplicación para realizar operaciones financieras cotidianas.

En Argentina, según datos del Instituto Nacional de Estadística y Censos (INDEC), aproximadamente el 10 % de la población presenta algún tipo de dificultad o limitación permanente [15]. Dentro de este universo, las limitaciones visuales y motoras representan una proporción significativa, lo que configura un segmento de usuarios que históricamente ha sido desatendido por el diseño centrado en la interfaz táctil. A nivel global, la Organización Mundial de la Salud (OMS) estima que más de 2.200 millones de personas presentan alguna forma de deficiencia visual, de las cuales una fracción importante se encuentra en edad económicamente activa y requiere acceso a servicios financieros [16]. La brecha entre la necesidad de acceso a servicios bancarios y la capacidad de las interfaces actuales para satisfacer dicha necesidad constituye un problema de equidad e inclusión que la tecnología tiene el potencial y la responsabilidad de resolver.

La motivación principal de este trabajo es derribar estas barreras tecnológicas, promoviendo una inclusión financiera real mediante el desarrollo de un canal de interacción alternativo. La implementación de un asistente transaccional por voz no solo genera un impacto social positivo al mejorar radicalmente la accesibilidad, sino que también ofrece una experiencia de usuario superior, simplificando operaciones complejas en un único paso conversacional y reduzca la fricción del proceso. Más aún, al garantizar que todo el procesamiento ocurra de forma local en el dispositivo, se elimina la resistencia que muchos usuarios podrían tener hacia el uso de comandos de voz en entornos de alta sensibilidad, como lo es la gestión de sus finanzas personales.

Capítulo 2

Introducción específica

El desarrollo de una solución basada en el procesamiento de lenguaje natural exige un acoplamiento profundo y eficiente con el hardware del dispositivo móvil. Este apartado detalla el ecosistema tecnológico y de software seleccionado para el desarrollo del prototipo.

2.1. Marco metodológico

Para la concepción, desarrollo y evaluación del sistema de transacciones financieras por voz, se adoptó como marco de trabajo la metodología CRISP-DM (*Cross-Industry Standard Process for Data Mining*) [17]. Si bien CRISP-DM fue concebida originalmente para proyectos de minería de datos, su estructura iterativa y centrada en el ciclo de vida de los datos la convierte en el estándar de facto para la ingeniería de proyectos de inteligencia artificial y machine learning. La elección de esta metodología responde a la necesidad de contar con un enfoque estructurado que permita alinear los objetivos técnicos del procesamiento de lenguaje natural con los requisitos críticos y las restricciones de seguridad propios de una aplicación de banca móvil. El ciclo de vida se ha adaptado específicamente para abordar el desarrollo de un pipeline NLU (*Natural Language Understanding*) [18] de ejecución local dentro del ecosistema iOS.

2.2. Herramientas utilizadas

Con el objetivo de cumplir con los requerimientos de procesamiento local (*Edge AI*) [19] y garantizar la privacidad por diseño (*Privacy by Design*) [20] en el contexto de una aplicación financiera, se adoptó de manera integral el entorno nativo de Apple. A continuación, se enumeran las herramientas integradas en la arquitectura del sistema.

- Swift [21]: es un lenguaje de programación compilado, multiparadigma y de tipado estático fuerte, desarrollado por Apple para la creación de software en sus sistemas operativos. Su adopción se justifica por su alto rendimiento computacional y su seguridad en el manejo de memoria mediante el Conteo Automático de Referencias (ARC). Estas características son esenciales para manipular flujos de datos en tiempo real (como el audio) y procesar estructuras de datos complejas sin incurrir en fugas de memoria o vulnerabilidades durante la ejecución transaccional.
- Xcode [22]: es el Entorno de Desarrollo Integrado (IDE) oficial y nativo provisto por Apple, que incluye los compiladores, depuradores y simuladores

necesarios para el ciclo de vida del software. Se utilizó como la herramienta central para la codificación, compilación y empaquetado del SDK. Su uso es mandatorio para desplegar la prueba de concepto en dispositivos físicos y para utilizar la herramienta *Instruments*, esta permitió perfilar y auditar el consumo de memoria y CPU de los modelos de Inteligencia Artificial durante la inferencia local.

- AVFoundation [23]: es el *framework* nativo de Apple que proporciona una interfaz de programación de bajo nivel para interactuar con el hardware audiovisual del dispositivo. Su uso fue indispensable para orquestar la captura segura del comando de voz. Específicamente, se utilizó la clase `AVAudioSession` para gestionar los permisos de privacidad del micrófono a nivel del sistema operativo, y la clase `AVAudioEngine` para interceptar la onda sonora cruda y aislar el flujo acústico sin depender de librerías de terceros.
- Speech Framework [24]: es una Interfaz de Programación de Aplicaciones (API) de Apple dedicada exclusivamente al Reconocimiento Automático de Habla (ASR), capaz de transcribir señales de audio a cadenas de texto. Se justificó su uso frente a otras alternativas comerciales debido a su capacidad de operar en modo *offline*. Al forzar la propiedad `requiresOnDevice-Recognition`, esta herramienta garantizó que la voz del usuario nunca fuera transmitida a servidores externos, mitigando los vectores de ataque y cumpliendo con el estándar de secreto bancario requerido por el proyecto.
- Core ML [25]: es el *framework* central de *Machine Learning* de Apple, diseñado para integrar, optimizar y ejecutar modelos predictivos directamente en el sistema operativo iOS. Constituye el motor analítico del proyecto. Se seleccionó porque delega automáticamente las cargas matemáticas pesadas (como la clasificación de intenciones y la extracción de entidades) al *Apple Neural Engine* (ANE). Esto garantiza que los algoritmos de Procesamiento de Lenguaje Natural (PLN) se ejecuten con una latencia mínima y un impacto energético marginal, habilitando el paradigma de inferencia 100 % *On-Device*.
- Coremltools [26]: es un paquete de código abierto basado en Python, desarrollado por Apple, que unifica la conversión de modelos entrenados en librerías estándar (como PyTorch o TensorFlow) al formato propietario `.mlpackage`. Su aplicación metodológica fue crítica para viabilizar el despliegue de las redes neuronales en el entorno móvil. Se utilizó para aplicar técnicas de cuantización a los pesos sinápticos de los modelos, reduciendo drásticamente su tamaño de almacenamiento y la huella en la memoria RAM del dispositivo, sin comprometer significativamente la exactitud predictiva del sistema.

2.3. Requerimientos y modelos de IA utilizados

El núcleo algorítmico del prototipo aborda la comprensión del comando de voz como un problema predictivo dual de machine learning: la clasificación de intenciones y el etiquetado de entidades transaccionales.

Por un lado, la clasificación multiclase a nivel de oración tiene como objetivo inferir la acción principal del usuario (por ejemplo, iniciar una transferencia o consultar un saldo). Matemáticamente, la red neuronal proyecta la secuencia de texto vectorizada hacia un espacio discreto de clases predefinidas, empleando una función de activación *softmax* para seleccionar, de manera excluyente, la intención que maximiza la probabilidad estadística.

Por otro lado, la conformación del *payload* financiero exige resolver un problema de etiquetado de secuencias a nivel de *token*, enmarcado en el Reconocimiento de Entidades Nombradas (NER)[27]. En lugar de emitir un único juicio para toda la oración, el modelo asigna una clase categórica a cada palabra utilizando el esquema de anotación IOB (*Inside*, *Outside*, *Beginning*). Esta técnica permite al sistema demarcar los límites exactos de variables críticas, como el monto numérico, la divisa y el destinatario, capturando las dependencias semánticas del lenguaje para extraer los parámetros exactos de la operación.

2.4. Exploración teórica de arquitecturas evaluadas

Para abordar este desafío predictivo dual y que se respeten las restricciones de memoria y procesamiento de los dispositivos móviles, se llevó a cabo una evaluación teórica de diversas arquitecturas algorítmicas, que analiza el compromiso (*trade-off*) entre precisión semántica, latencia de inferencia y consumo de recursos.

En primer lugar, se analizó el estado del arte representado por las arquitecturas basadas en Transformers. Este paradigma, fundamentado en el mecanismo de autoatención, prescinde de la recurrencia secuencial y permite capturar dependencias a largo plazo y procesa el contexto de manera bidireccional y paralelizable. Dentro de esta familia, se evaluó la viabilidad de incorporar modelos del lenguaje preentrenados adaptados al español, como BETO (la variante hispanohablante de BERT) [28]. Si bien BETO ofrece un rendimiento superior en la comprensión de las sutilezas léxicas y morfológicas del lenguaje coloquial rioplatense, su implementación nativa en el edge presenta obstáculos significativos. La alta dimensionalidad de sus parámetros (típicamente superior a 110 millones) exige la aplicación de técnicas agresivas de compresión, poda (*pruning*) y destilación de conocimiento (*knowledge distillation*) para transformarlo en un artefacto viable para el entorno iOS sin agotar la memoria RAM del dispositivo.

Como alternativa de menor huella computacional, se exploró la familia de las redes neuronales recurrentes (RNN) [29]. A diferencia de los transformers, las RNN procesan los tokens de manera estrictamente secuencial, actualiza un estado oculto que actúa como memoria temporal del contexto lingüístico. Dado que las arquitecturas RNN tradicionales son susceptibles al problema del desvanecimiento del gradiente (*vanishing gradient*) al procesar secuencias largas, la investigación se centró en sus variantes con mecanismos de compuerta (*gating*): las redes de memoria a corto y largo plazo (LSTM) [30] y las unidades recurrentes con compuerta (GRU) [31]. Estas topologías regulan matemáticamente el flujo de información, dice qué características del contexto previo deben retenerse y cuáles descartarse. Históricamente, las arquitecturas basadas en LSTM bidireccionales (BiLSTM) [32] han demostrado una alta eficacia para el etiquetado de secuencias (NER) [27], logra una precisión competitiva con un costo computacional y tamaño de modelo órdenes de magnitud inferiores a los de un modelo fundacional, lo que las hace inherentemente más compatibles con inferencias On-Device [33].

Finalmente, se evaluó la pertinencia de utilizar los frameworks de comprensión de lenguaje natural nativos de Apple, en particular la API NaturalLanguage y el ecosistema de Create ML [34]. Apple proporciona clases altamente especializadas, como `NLModel` y `NLTagger`, diseñadas para compilar clasificadores de texto y extractores de entidades a partir de *datasets* etiquetados. La ventaja superlativa de este enfoque reside en su integración opaca y de bajo nivel con el silicio del dispositivo. Los modelos generados bajo este estándar están nativamente optimizados para aprovechar la aceleración por hardware del procesamiento neural de Apple, garantiza tiempos de latencia del orden de los milisegundos y un impacto insignificante en la autonomía de la batería. Aunque la utilización de frameworks cerrados sacrifica el grado de control arquitectónico y la hiperparametrización profunda que ofrecen librerías como PyTorch o TensorFlow, su robustez operativa, la garantía de privacidad al aislar el procesamiento de la red, y la facilidad de integración mediante Core ML, posicionan a estas herramientas nativas como candidatos estratégicos para validar la viabilidad técnica de una prueba de concepto en el sector financiero.

A continuación se detallan las métricas que respaldan lo mencionado en el presente capítulo:

Criterio	BETO (Transformer)	BiLSTM (RNN)	Create ML (MaxEnt)	Create ML (CRF)
Tamaño del modelo (MB)	~440	~5–15	<1	<1
Parámetros (millones)	~110	~0.5–2	N/A	N/A
Latencia de inferencia (ms)	>200	20–50	<10	<10
Memoria RAM en runtime (MB)	>300	15–40	<5	<5
Requiere conversión Core ML	Sí	Sí	No	No
Aceleración Neural Engine	Parcial	Parcial	Nativa	Nativa
Entrenamiento sin GPU	No viable	Posible	Sí (CPU)	Sí (CPU)
Privacidad <i>on-device</i>	Sí	Sí	Sí	Sí

TABLA 2.1. Comparativa de arquitecturas evaluadas para NLU *on-device* en iOS. Los valores de BETO y BiLSTM corresponden a referencias bibliográficas; los de Create ML (MaxEnt/CRF) son resultados experimentales propios.

Capítulo 3

Implementación y desarrollo

Este capítulo expone el proceso técnico de implementación y desarrollo del sistema propuesto. En primer lugar, se describe la arquitectura de la solución, instancia en la que se definió la separación de responsabilidades entre la aplicación base y el SDK nativo denominado Aurum. A continuación, se detalla el modelo de datos estructurado que se diseñó para encapsular las intenciones y entidades extraídas de los comandos de voz. Finalmente, se explican los componentes del código fuente del prototipo.

3.1. Arquitectura del sistema

A continuación, se detallan cada una de las fases técnicas atravesadas para la elaboración de este trabajo.

- **Comprensión del negocio:** esta fase inicial constituye el pilar fundamental del trabajo, dado que define el puente entre la necesidad de accesibilidad del usuario y las exigencias transaccionales del sistema bancario. El problema central del negocio radica en las barreras de usabilidad que las interfaces táctiles tradicionales imponen a personas con discapacidades visuales o motoras. La solución propuesta es habilitar un canal conversacional.

Desde la perspectiva del sistema, la comprensión del negocio se traduce en el desafío algorítmico de transformar lenguaje natural, inherentemente ambiguo y desestructurado, en formatos estructurados y accionables que el core bancario pueda procesar de manera segura. Por ejemplo, una expresión coloquial como "transferile cinco mil pesos a mamá" carece de utilidad directa para una base de datos financiera. El objetivo del negocio exige que el pipeline de IA sea capaz de capturar esta emisión acústica y extraer inequívocamente una intención principal (Clasificación de Intención: TRANSFERENCIA) y los parámetros o entidades asociadas (Reconocimiento de Entidades Nombradas: MONTO: 5000, MONEDA: ARS, DESTINATARIO: mamá). Solamente al mapear el habla hacia este diccionario de datos estructurado (típicamente en formato JSON o estructuras nativas de Swift), el sistema móvil puede invocar la lógica de negocio correspondiente, lo que garantiza una operación determinista, libre de ambigüedades y sin fricción para el usuario.

- **Comprensión de los datos:** una vez definido el objetivo de extraer intenciones y entidades transaccionales, esta fase se centra en identificar la naturaleza y los requerimientos de la información necesaria para entrenar el

modelo. En el contexto de la banca móvil, los datos reales de los usuarios están protegidos por estrictas normativas de privacidad y secreto bancario. Al tratarse de una prueba de concepto y ante la ausencia de un registro histórico de comandos de voz accionados por usuarios, se determinó la necesidad de construir un conjunto de datos (*dataset*) sintético. En esta etapa se analizó la morfología del lenguaje coloquial rioplatense aplicado a finanzas, identifica sinónimos de acciones (transferir, enviar, pasar plata), variaciones en la expresión de montos (números cardinales, fracciones) y formas comunes de nombrar contactos.

- **Evaluación:** esta fase contrastó los resultados empíricos del modelo contra los criterios de éxito definidos al inicio del trabajo. A nivel algorítmico, se midió el desempeño predictivo que utiliza un conjunto de prueba aislado, que calcula métricas de Accuracy para la clasificación de intenciones, así como Precision, Recall y F1-Score para el reconocimiento de entidades. A nivel de negocio e integración, la evaluación trascendió las métricas puramente matemáticas para incorporar pruebas de usabilidad en el entorno simulado de iOS, mide la latencia de la inferencia local y la tasa de éxito de la tarea integral. Esto permitió verificar que la extracción estructurada lograra, en efecto, completar el flujo de la aplicación sin errores perjudiciales para la experiencia del usuario.
- **Despliegue:** en un proyecto de analítica tradicional, el despliegue suele implicar la exposición del modelo a través de una API en un servidor en la nube. Sin embargo, en esta prueba de concepto, la fase de despliegue se redefinió para cumplir con el requisito innegociable de máxima privacidad. El despliegue consistió en la conversión de los modelos entrenados al formato `.mlpackage`, su integración directa en el código fuente del proyecto en Xcode, y su invocación local mediante el framework Core ML. De esta manera, el modelo se "desplegó" encapsulado dentro del propio binario de la aplicación iOS, se garantiza que el pipeline de transformación de voz a texto (vía AVFoundation y Speech) y la posterior comprensión del lenguaje natural se ejecuten de manera estrictamente local en el dispositivo del usuario final.

El diseño arquitectónico de la presente prueba de concepto se fundamenta en el paradigma de computación Edge AI, trasladando la carga computacional de los algoritmos de inteligencia artificial directamente al dispositivo móvil del usuario. Esta decisión estratégica persigue un doble objetivo metodológico y normativo: por un lado, minimizar la latencia inherente a las comunicaciones de red corporativas y, por otro, garantizar un entorno de privacidad por diseño. Al ejecutar los modelos a través del *framework* Core ML, los flujos de audio y las transcripciones de índole financiera no abandonan en ningún momento el entorno físico del hardware de Apple, lo que mitiga drásticamente los vectores de ataque asociados a la interceptación de datos en tránsito o al almacenamiento temporal en servidores de terceros. Para asegurar la escalabilidad, la modularidad y la mantenibilidad del código, el sistema fue concebido bajo el principio de separación de responsabilidades. La solución se estructura en dos grandes bloques funcionales e independientes: la aplicación base anfitriona y el SDK nativo Aurum [35]. Es imperativo establecer una restricción arquitectónica crítica que define los límites de esta integración: el proyecto no incluye, bajo ninguna circunstancia, algoritmos de validación biométrica de voz. La autenticación de la identidad del cliente, el manejo de tokens de sesión y las medidas de seguridad perimetral recaen, de

manera exclusiva y excluyente, sobre la aplicación móvil del banco. En este modelo topológico, el SDK Aurum opera bajo la premisa de confianza de que es invocado dentro de un contexto transaccional previamente securizado por la entidad financiera, lo que hace adoptar el dominio de responsabilidad estrictamente a la captura acústica, la transcripción y la extracción semántica de intenciones y entidades.

En la figura 3.1 se presenta el diagrama de componentes.

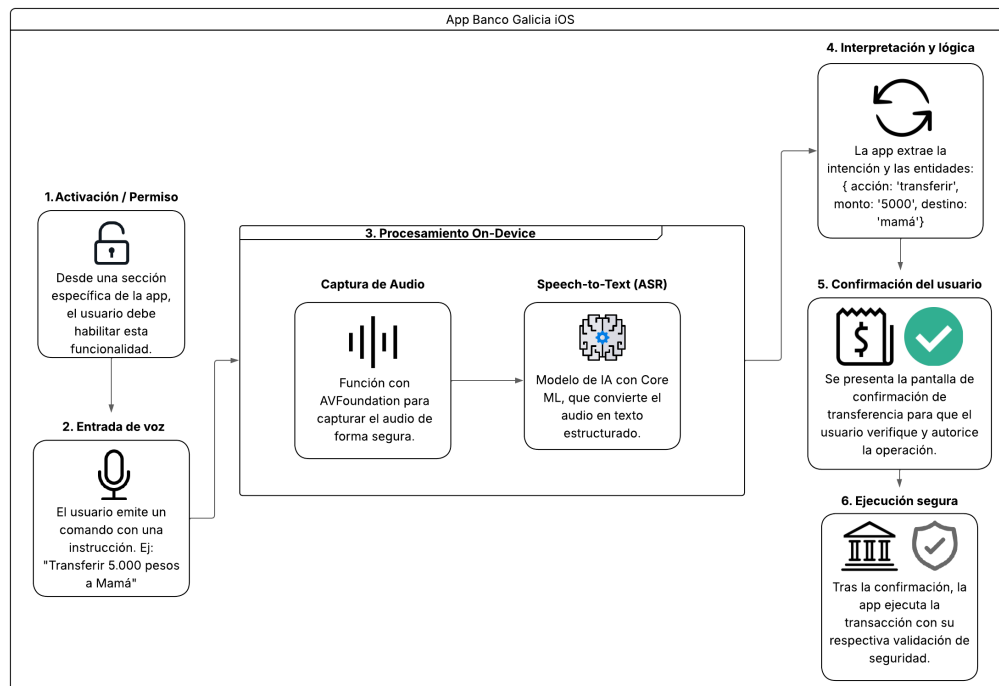


FIGURA 3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.

El ciclo de vida de una transacción impulsada por voz atraviesa un pipeline determinista que fluye secuencialmente a través de las capas de la arquitectura descrita. El proceso de ingesta de datos se desencadena en la aplicación base cuando el usuario, luego de superar los controles de acceso convencionales del banco, interactúa con el disparador de la interfaz gráfica (botón de dictado). En ese instante, la aplicación anfitriona invoca la API pública del SDK Aurum, transfiriéndole el control del hilo de ejecución de audio. El SDK inicializa una sesión de captura de alto rendimiento mediante el framework AVFoundation, configurando el entorno acústico adecuado para aislar la frecuencia de la voz humana. Una vez digitalizada la señal de audio, el motor interno del SDK emplea las capacidades de transcripción fuera de línea (offline) provistas por las librerías nativas de iOS para ejecutar el reconocimiento automático de habla, esto convierte el espectrograma en una cadena de texto cruda. Acto seguido, este texto es sometido a técnicas de normalización morfológica y es inyectado en el motor de comprensión del lenguaje natural. En esta fase de inferencia, la transcripción es procesada localmente por los modelos vectoriales preentrenados y cuantizados alojados en Core ML. Dichos modelos resuelven algorítmicamente el problema de clasificación de la intención del usuario y proceden al etiquetado posicional para extraer las variables nominales del comando. El flujo de información concluye cuando

la salida matricial de las redes neuronales es traducida por el SDK en un payload [36] de datos estructurado y fuertemente tipado. Este artefacto resultante, que encapsula inequívocamente la acción solicitada, la magnitud numérica del monto, el tipo de divisa y la identidad del beneficiario, es retornado de manera asíncrona a la aplicación anfitriona a través de patrones de delegación (*Delegates* o *Closures*)[21]. A partir de la entrega de este objeto estructurado, el SDK Aurum finaliza su intervención y cede nuevamente el control a la aplicación base que asume la responsabilidad de mapear el payload contra su modelo de datos transaccional, renderizar la interfaz de validación final ante el usuario y proceder con la comunicación cifrada hacia el core bancario.

En la figura 3.2 se muestra el flujo del pipeline (ciclo de vida de una transacción por voz).

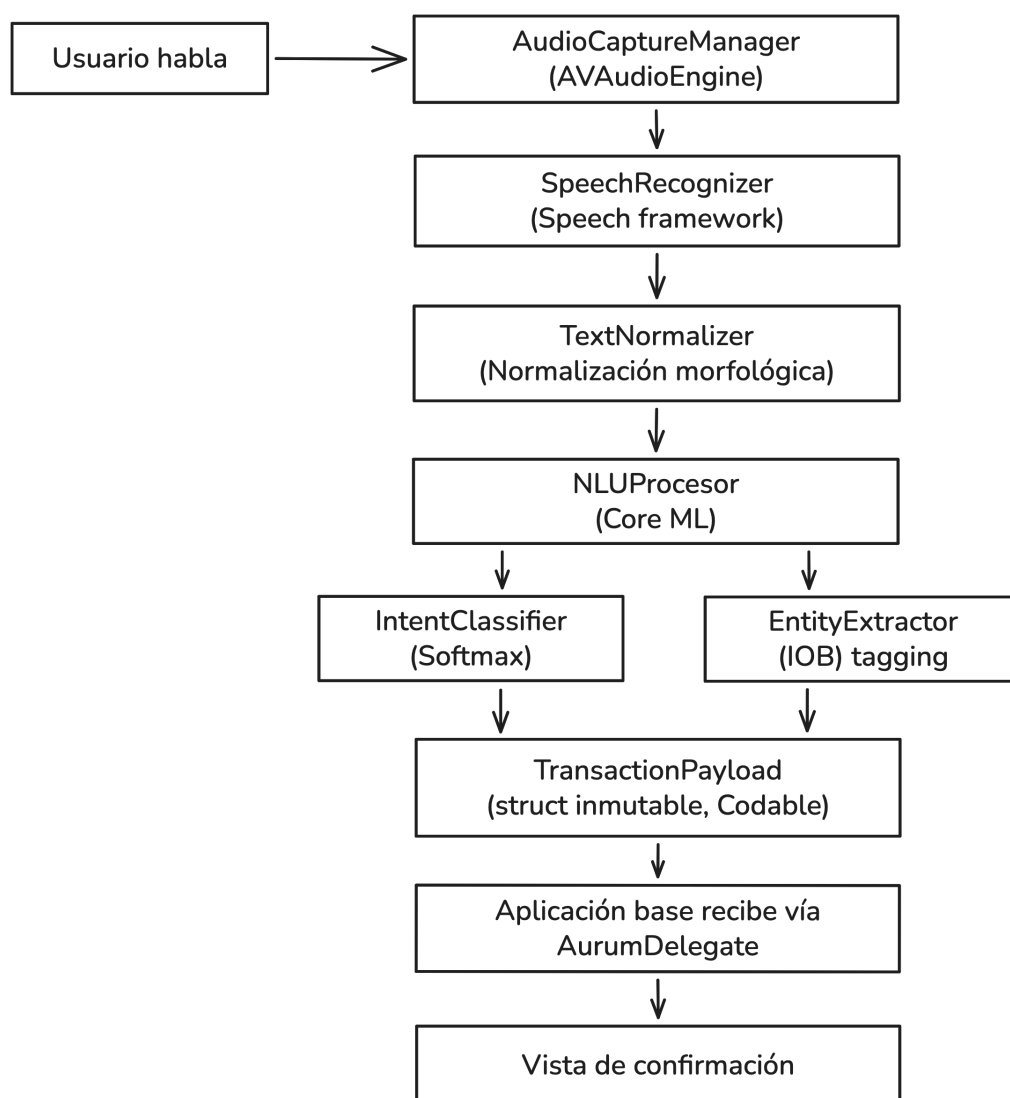


FIGURA 3.2. Flujo de pipeline de una transacción realizada por voz.

3.2. Tratamiento y preparación de datos

El desarrollo de modelos de procesamiento de lenguaje natural aplicados a dominios de misión crítica, como el sector bancario, enfrenta el desafío inherente del acceso a la información, conocido como el problema de arranque en frío (*cold start*). Las estrictas normativas de privacidad y el secreto bancario imposibilitan el uso de corpus conversacionales de clientes reales sin someterlos previamente a procesos de anonimización complejos y costosos. Adicionalmente, al tratarse de una prueba de concepto que introduce un canal de interacción inédito en el entorno simulado de la aplicación móvil, no existe telemetría histórica ni registros acústicos o textuales previos de comandos transaccionales. Ante esta restricción, se determinó como exigencia metodológica la generación de un conjunto de datos sintéticos (*mock data*). La construcción de un corpus artificial permite no solo superar la ausencia de datos iniciales, sino también ejercer un control riguroso sobre la distribución probabilística de las clases (intenciones) y garantizar una cobertura exhaustiva de los casos límite (*edge cases*) de las entidades. Este enfoque metodológico aísla el desarrollo algorítmico de los riesgos de privacidad y provee un entorno determinista, fundamental para iterar la arquitectura de la red neuronal durante las primeras fases de evaluación.

3.2.1. Creación de plantillas de comandos y aumentación de datos

La síntesis del corpus de entrenamiento se estructuró a partir de la definición de plantillas (*templates*) gramaticales. Estas plantillas modelan lógicamente las posibles estructuras sintácticas que un usuario emplearía para interactuar con el sistema bancario y se abstraen los valores concretos, tales como: [ACCIÓN] [MONTO] [MONEDA] a [DESTINATARIO]. Una vez consolidadas las estructuras base, se implementaron técnicas de aumentación de datos (*data augmentation*) para inyectar la variabilidad inherente al lenguaje natural [37].

La transición entre el entorno probabilístico inherente a los modelos de machine learning y el rigor determinista exigido por los sistemas financieros requiere de una capa de abstracción robusta. Para resolver este paradigma, se estructuraron los datos resultantes de la inferencia algorítmica utilizando estructuras de valor *struct* en Swift [21]. El uso de *structs* garantizó la inmutabilidad de la información una vez extraída y procesada, esto previene alteraciones accidentales de las variables en la memoria durante el ciclo de vida de la transacción. Para la serialización de este modelo, se adoptó el protocolo Codable nativo de Apple [38]. Esta decisión arquitectónica se justificó en la necesidad de estandarizar la comunicación entre el SDK y la aplicación base, así el objeto Swift generado en memoria puede ser codificado y decodificado de manera nativa hacia formatos universales de intercambio de datos, como JSON [39], sin depender de librerías de terceros.

3.2.2. Modelo de datos

El diseño del payload (o carga útil) que el modelo de etiquetado de secuencias (NER) y el clasificador de intenciones devolvieron al sistema se consolidó en una única entidad semántica. Durante el desarrollo, se definieron campos fuertemente tipados para encapsular de manera exacta las variables transaccionales críticas:

- **Intención:** representa la acción directiva inferida del comando verbal (por ejemplo, iniciar una transferencia). Se estructuró como el eje central que define el ruteo posterior dentro de la aplicación.
- **Monto:** se tipificó mediante el uso de datos de punto flotante de doble precisión (*Double*) [21], esto permite la representación matemática de la magnitud monetaria extraída de la transcripción fonética.
- **Moneda:** se modeló utilizando enumeraciones (*Enum*) con tipos crudos de cadena de texto (*String raw values*) [21]. Esto restringió el dominio de la divisa a valores finitos y normalizados bajo estándares internacionales (ej. ARS o USD), eliminando la ambigüedad de sinónimos coloquiales capturados por el motor NLU, como "pesos", "lucas" o "dólares".
- **Destinatario:** se definió como una cadena de caracteres cruda (*String*) [21] que encapsula la identidad nominal del beneficiario de la operación, dejando el valor preparado para ser contrastado posteriormente contra la agenda de contactos o base de alias validada del banco.

Este enfoque de modelado determinista cumple una función vital en la arquitectura de integración del sistema. Al transformar una emisión acústica originalmente desestructurada en un objeto Swift predecible y validado a nivel de compilador, se abstraigo a la aplicación anfitriona de toda la complejidad inherente al procesamiento de lenguaje natural. En consecuencia, la integración con los servicios de backend del banco se facilita de manera significativa: la aplicación móvil anfitriona consume un payload limpio, cuya estructura es análoga a la que generaría una interfaz táctil tradicional basada en formularios. Esto habilita a la aplicación base a tomar dichas variables tipadas e inyectarlas de manera directa en los constructores de sus peticiones de red seguras, esto mantendría intacta la lógica de consumo de los endpoints transaccionales de la institución.

3.3. Desarrollo del SDK (Aurum)

El desarrollo de Aurum se abordó bajo el paradigma de la programación orientada a componentes y se materializó como un *framework* dinámico reutilizable dentro del ecosistema de Apple. El propósito fundamental de este encapsulamiento fue abstraer la complejidad algorítmica inherente al procesamiento de señales acústicas y a la inferencia de modelos de *machine learning*, proporcionando a la aplicación bancaria anfitriona una interfaz programática mínima que oculta la totalidad de las operaciones internas. Al aislar el *pipeline* de inteligencia artificial en un módulo independiente, se garantizó la cohesión del código, se facilitó su distribución y mantenimiento futuro, y se evitó la contaminación de la lógica de negocio y de la interfaz de usuario de la aplicación consumidora.

La arquitectura interna del SDK se organiza en cinco componentes especializados, cada uno con una responsabilidad bien delimitada, que se ejecutan de forma secuencial y coordinada bajo la orquestación de una clase fachada principal. La tabla 3.1 presenta un resumen de estos componentes y sus dependencias de frameworks nativos de Apple.

Las secciones siguientes describen el diseño e implementación de cada componente, así como la API pública expuesta a la aplicación consumidora.

Componente	Clase	Framework	Responsabilidad
Fachada principal	AurumEngine	—	Orquestación del pipeline
Captura de audio	AudioCaptureManager	AVFoundation	Configuración de sesión y captura de buffers
Reconocimiento de voz	SpeechRecognizer	Speech	Transcripción <i>on-device</i>
Normalización de texto	TextNormalizer	Foundation	Conversión léxica y numérica
Procesamiento NLU	NLUProcessor	NaturalLanguage, CoreML	Clasificación de intención y extracción de entidades

TABLA 3.1. Componentes del SDK Aurum y sus dependencias de frameworks nativos.

3.3.1. Clase fachada: AurumEngine

Para orquestar la ejecución secuencial de las etapas de procesamiento, se implementó la clase `AurumEngine` siguiendo el patrón de diseño *Facade* [40]. Esta clase constituye el único punto de entrada público del SDK y su responsabilidad es coordinar el ciclo de vida integral de la transacción de voz: desde la activación del micrófono hasta la entrega del `TransactionPayload` resultante.

Internamente, `AurumEngine` mantiene referencias privadas a los cuatro componentes de procesamiento y los instancia durante la fase de configuración. La API pública expone únicamente tres métodos de control:

```

1 public class AurumEngine {
2
3     public weak var delegate: AurumDelegate?
4
5     private let audioManager = AudioCaptureManager()
6     private let speechRecognizer = SpeechRecognizer()
7     private let textNormalizer = TextNormalizer()
8     private let nluProcessor = NLUProcessor()
9
10    /// Precarga los modelos Core ML desde el bundle de la app.
11    public func configure(bundle: Bundle) {
12        nluProcessor.loadModels(from: bundle)
13    }
14
15    /// Inicia la captura de audio y el reconocimiento de voz.
16    public func startListening() { /* ... */ }
17
18    /// Detiene la captura y fuerza el procesamiento del texto acumulado.
19    public func stopListening() { /* ... */ }
20
21    /// Cancela la operacion en curso y libera recursos.
22    public func cancelListening() { /* ... */ }
23 }

```

CÓDIGO 3.1. API pública de `AurumEngine`.

El método `configure(bundle:)` recibe el `Bundle` de la aplicación anfitriona para localizar los archivos `.mlmodelc` compilados (`IntentClassifier` y

`EntityExtractor`) e instanciar los objetos `NLModel` correspondientes. Esta pre-carga se ejecuta una única vez durante el inicio de la aplicación y permite que las inferencias posteriores se resuelvan con latencia mínima, evitando el costo de carga en caliente (*cold start*) durante la interacción de voz.

3.3.2. Captura de audio: `AudioCaptureManager`

El primer eslabón del pipeline es la gestión del hardware de audio del dispositivo. El componente `AudioCaptureManager` encapsula la interacción con los frameworks `AVAudioSession` y `AVAudioEngine` de Apple, y resuelve tres tareas técnicas:

1. Solicitud de permisos: la primera invocación verifica el estado de autorización del micrófono mediante `AVAudioSession.recordPermission`. Si el permiso no fue concedido previamente, dispara la solicitud nativa del sistema operativo. En caso de denegación, el componente reporta un error tipado (`AurumError.microphonePermissionDenied`) sin intentar activar la captura.
2. Configuración de la sesión de audio: se establece la categoría `.playAndRecord` con modo `.measurement`. La categoría `.playAndRecord` permite la entrada y salida simultánea de audio (necesaria para dispositivos con altavoz activo), mientras que el modo `.measurement` desactiva los filtros de ecualización y cancelación de ruido del sistema, proporcionando una señal de audio cruda óptima para el reconocedor de voz.
3. Instalación del *tap* de audio: se accede al nodo de entrada (`inputNode`) del `AVAudioEngine` y se instala un *tap* que captura *buffers* de audio en formato PCM a la frecuencia de muestreo nativa del micrófono (típicamente 44.1 kHz o 48 kHz). Estos *buffers* se canalizan al componente `Speech Recognizer` mediante un cierre (*closure*).

```

1 func startCapture(onBuffer: @escaping (AVAudioPCMBuffer) -> Void) throws
2 {
3     let session = AVAudioSession.sharedInstance()
4     try session.setCategory(.playAndRecord, mode: .measurement,
5                             options: [.defaultToSpeaker])
6     try session.setActive(true)
7
8     let inputNode = audioEngine.inputNode
9     let format = inputNode.outputFormat(forBus: 0)
10
11     inputNode.installTap(onBus: 0, bufferSize: 1024, format: format)
12     { buffer, _ in
13         onBuffer(buffer)
14     }
15
16     audioEngine.prepare()
17     try audioEngine.start()
18 }

```

CÓDIGO 3.2. Configuración de la sesión de audio y captura de *buffers*.

La detención de la captura (`stopCapture()`) remueve el *tap*, detiene el `AVAudioEngine` y desactiva la sesión de audio, liberando el hardware del micrófono para que otras aplicaciones puedan utilizarlo.

3.3.3. Reconocimiento de voz: `SpeechRecognizer`

El componente `SpeechRecognizer` integra el *framework* nativo `Speech` [24] para ejecutar la transcripción de voz a texto. Su diseño responde a dos requisitos fundamentales del sistema: el soporte para español rioplatense y la garantía de procesamiento exclusivamente local.

La instanciación del reconocedor se realiza con el *locale* específico `es-AR` (español de Argentina), lo que activa el modelo acústico y el modelo de lenguaje optimizados para las particularidades fonéticas y léxicas del habla rioplatense:

```

1 private let recognizer = SFSpeechRecognizer(
2     locale: Locale(identifier: "es-AR")
3 )
4
5 func startRecognition(onPartial: @escaping (String) -> Void,
6                     onFinal: @escaping (String) -> Void) {
7     let request = SFSpeechAudioBufferRecognitionRequest()
8
9     // Forzar procesamiento local: ningun audio sale del dispositivo
10    request.requiresOnDeviceRecognition = true
11
12    // Sugerencias contextuales para el dominio financiero
13    request.contextualStrings = [
14        "transferir", "transferencia", "enviar", "mandar",
15        "consultar", "saldo", "cancelar", "anular",
16        "pesos", "dolares", "lucas", "verdes", "mangos"
17    ]
18
19    recognizer?.recognitionTask(with: request) { result, error in
20        guard let result = result else { return }
21        let text = result.bestTranscription.formattedString
22        if result.isFinal {
23            onFinal(text)
24        } else {
25            onPartial(text)
26        }
27    }
28 }

```

CÓDIGO 3.3. Configuración del reconocedor de voz con procesamiento *on-device*.

La propiedad `requiresOnDeviceRecognition = true` es la piedra angular de la estrategia de privacidad del sistema. Esta configuración instruye al *Speech Framework* a utilizar exclusivamente el modelo de reconocimiento descargado localmente en el dispositivo, bloqueando cualquier transmisión de paquetes de audio a los servidores de Apple. Si el modelo local no se encuentra disponible (por ejemplo, porque el usuario no descargó el paquete de idioma `texttts-AR` para uso sin conexión), el reconocedor reporta un error en lugar de recurrir silenciosamente al procesamiento en la nube.

La propiedad `contextualStrings` proporciona al motor de reconocimiento un vocabulario de refuerzo específico del dominio financiero. Al incluir términos como “transferencia”, “saldo” o “lucas”, se incrementa la probabilidad de que el reconocedor transcriba correctamente estas palabras frente a alternativas fonéticamente similares pero semánticamente irrelevantes. Este mecanismo de *biasing* léxico no modifica el modelo acústico subyacente, sino que ajusta las probabilidades del modelo de lenguaje durante la decodificación.

El reconocimiento opera en modo *streaming*: a medida que el usuario habla, el *framework* genera hipótesis parciales que se entregan progresivamente al delegado de la app para su visualización en tiempo real. Cuando el motor detecta una pausa prolongada o el final del enunciado, emite el resultado final (`result.isFinal = true`) que constituye la entrada para la etapa de normalización.

3.3.4. Normalización de texto: `TextNormalizer`

La transcripción cruda del reconocedor de voz contiene frecuentemente expresiones coloquiales, numéricas y morfológicas que no pueden ser procesadas directamente por los modelos de clasificación e extracción de entidades. El componente `TextNormalizer` actúa como capa intermedia de limpieza y estandarización, transformando el texto transcrito en una forma canónica compatible con el vocabulario del corpus de entrenamiento.

Las transformaciones implementadas abarcan tres categorías:

Conversión de expresiones numéricas

El normalizador convierte las representaciones textuales de números a su forma numérica. Esta conversión abarca tanto las formas estándar del español (“quinientos” → 500, “mil” → 1000, “doscientos cincuenta” → 250) como las expresiones coloquiales rioplatenses que involucran multiplicadores implícitos.

Normalización de léxico coloquial

El habla rioplatense utiliza un conjunto de términos coloquiales para referirse a monedas y denominaciones que no forman parte del vocabulario financiero formal. El normalizador mantiene un diccionario de equivalencias que mapea estas expresiones a sus formas canónicas:

```

1 private let colloquialMap: [String: (canonical: String, multiplier:
    Double?)] = [
2     // Denominaciones de moneda
3     "lucas": (canonical: "pesos", multiplier: 1000),
4     "luca": (canonical: "pesos", multiplier: 1000),
5     "mangos": (canonical: "pesos", multiplier: nil),
6     "guita": (canonical: "pesos", multiplier: nil),
7     "verdes": (canonical: "dolares", multiplier: nil),
8
9     // Expresiones de cantidad
10    "palo": (canonical: "pesos", multiplier: 1_000_000),
11    "gambas": (canonical: "pesos", multiplier: 100),
12 ]

```

CÓDIGO 3.4. Diccionario de normalización para léxico coloquial rioplatense.

Cuando el normalizador encuentra la expresión “dos lucas”, la resuelve como $2 \times 1000 = 2000$ pesos: el numeral “dos” se convierte a su valor numérico, el término “lucas” aporta la moneda canónica (“pesos”) y el multiplicador ($\times 1000$). Análogamente, “cinco gambas” se transforma en $5 \times 100 = 500$ pesos, y “medio palo” en $0.5 \times 1\,000\,000 = 500\,000$ pesos.

Normalización morfológica

Esta categoría comprende la conversión de formas verbales del voseo rioplatense a sus formas imperativas estándar (“transferí” → “transferir”, “fijate” → “consultar”) y la eliminación de partículas coloquiales que no aportan contenido semántico (“che”, “dale”, “na”, “bueno”). Estas transformaciones reducen la variabilidad léxica de la entrada y mejoran la probabilidad de que los modelos clasifiquen correctamente la intención, dado que el corpus de entrenamiento incluye predominantemente formas canónicas.

3.3.5. Procesamiento NLU: NLUProcessor

El componente `NLUProcessor` constituye el núcleo de inteligencia artificial del SDK. Su responsabilidad es resolver el problema dual de clasificación de intenciones y extracción de entidades nombradas (NER) a partir del texto normalizado, utilizando los modelos Core ML entrenados con el *framework* Create ML.

La arquitectura del procesador se basa en las clases `NLModel` y `NLTagger` del *framework* `NaturalLanguage` de Apple [41], que proporcionan una interfaz de alto nivel para la inferencia de modelos compilados en formato `.mlmodelc`.

Clasificación de intenciones

La clasificación de intenciones determina cuál de las tres operaciones transaccionales soportadas corresponde al comando de voz del usuario. El modelo `IntentClassifier`, entrenado con el algoritmo *Maximum Entropy* (MaxEnt), se carga como instancia de `NLModel` y expone un método de predicción que recibe un *string* y retorna la etiqueta de clase con mayor probabilidad junto con su nivel de confianza:

```

1 private var intentModel: NLModel?
2
3 func classifyIntent(_ text: String) -> (TransactionIntent, Float) {
4     guard let model = intentModel else {
5         return (.desconocida, 0.0)
6     }
7
8     let hypotheses = model.predictedLabelHypotheses(
9         for: text, maximumCount: 3
10    )
11
12    guard let topLabel = model.predictedLabel(for: text),
13          let confidence = hypotheses[topLabel] else {
14        return (.desconocida, 0.0)
15    }
16
17    // Aplicar umbral de confianza
18    guard confidence >= intentConfidenceThreshold else {
19        return (.desconocida, Float(confidence))
20    }
21
22    let intent = TransactionIntent(rawValue: topLabel) ?? .desconocida
23    return (intent, Float(confidence))
24 }
```

CÓDIGO 3.5. Clasificación de intención con umbral de confianza.

El umbral de confianza (`intentConfidenceThreshold = 0.65`) actúa como mecanismo de seguridad: si ninguna de las tres clases alcanza este valor, el procesador asigna la intención `.desconocida`, lo que evita que el sistema ejecute una operación financiera basada en una clasificación incierta. El valor de 0.65 fue seleccionado empíricamente como punto de equilibrio entre la tasa de rechazo de comandos legítimos (*false negatives*) y la aceptación errónea de comandos ambiguos (*false positives*).

Las tres intenciones soportadas se modelan mediante un enumerado tipado:

```

1 enum TransactionIntent: String, Codable {
2     case transferencia = "TRANSFERENCIA"
3     case consultaSaldo = "CONSULTA_SALDO"
4     case cancelar      = "CANCELAR"
5     case desconocida   = "DESCONOCIDA"
6
7     var requiresEntityExtraction: Bool {
8         switch self {
9             case .transferencia: return true
10            case .consultaSaldo, .cancelar, .desconocida: return false
11        }
12    }
13 }

```

CÓDIGO 3.6. Enumerado de intenciones transaccionales.

La propiedad computada `requiresEntityExtraction` permite optimizar el pipeline: si la intención detectada es `.consultaSaldo` o `.cancelar`, se omite la etapa de extracción de entidades (que solo es relevante para transferencias), reduciendo la latencia total del procesamiento.

Extracción de entidades nombradas (NER)

Para las intenciones de tipo `.transferencia`, el procesador activa la extracción de entidades nombradas utilizando el modelo `EntityExtractor`, entrenado con el algoritmo *Conditional Random Field* (CRF). Este modelo, cargado en un `NLTagger`, etiqueta cada token del texto normalizado con una de las siete clases del esquema IOB definido: B-MONTO, I-MONTO, B-MONEDA, I-MONEDA, B-DESTINATARIO, I-DESTINATARIO y O (*Outside*).

```

1 private var entityTagger: NLTagger?
2
3 func extractEntities(_ text: String) -> (Double?, Currency?, String?) {
4     guard let tagger = entityTagger else { return (nil, nil, nil) }
5
6     tagger.string = text
7     var currentEntity: String? = nil
8     var entityTokens: [String: [String]] = [:]
9
10    tagger.enumerateTags(
11        in: text.startIndex..

```

```

20         currentEntity = String(label.dropFirst(2))
21         entityTokens[currentEntity!, default: []].append(token)
22     } else if label.hasPrefix("I-"), let entity = currentEntity {
23         entityTokens[entity, default: []].append(token)
24     } else {
25         currentEntity = nil
26     }
27     return true
28 }
29
30 // Reconstruir valores tipados
31 let amount = parseAmount(entityTokens["MONTO"])
32 let currency = Currency.fromTokens(entityTokens["MONEDA"])
33 let recipient = entityTokens["DESTINATARIO"]?.joined(separator: " ")
34
35 return (amount, currency, recipient)
36 }

```

CÓDIGO 3.7. Extracción de entidades con NLTagger y esquema IOB.

La reconstrucción de entidades a partir de las etiquetas IOB sigue un algoritmo lineal: al encontrar una etiqueta B-X se inicia una nueva entidad de tipo X; las etiquetas I-X consecutivas extienden la entidad actual; cualquier otra etiqueta cierra la entidad en curso. Este esquema permite capturar entidades de longitud variable, como nombres compuestos ("Juan Pérez" = B-DESTINATARIO + I-DESTINATARIO) o montos con modificadores ("dos mil quinientos" = B-MONTO + I-MONTO + I-MONTO).

Las tres entidades extraídas se tipan mediante las estructuras de datos del SDK: MONTO se convierte a `Double`, MONEDA se mapea al enumerado `Currency` (que soporta las variantes coloquiales mediante el método estático `fromColloquial`), y DESTINATARIO se reconstruye como `String` concatenando los tokens etiquetados.

3.3.6. Modelo de datos: `TransactionPayload`

El resultado consolidado del pipeline se encapsula en la estructura `TransactionPayload`, un *value type* inmutable que adopta los protocolos `Codable` y `Equatable` de Swift:

```

1 struct TransactionPayload: Codable, Equatable {
2     let intent: TransactionIntent
3     let amount: Double?
4     let currency: Currency?
5     let recipient: String?
6     let rawTranscription: String
7     let intentConfidence: Float
8     let timestamp: Date
9 }

```

CÓDIGO 3.8. Estructura del *payload* de transacción.

La elección de una `struct` (tipo por valor) en lugar de una `class` (tipo por referencia) es deliberada: al ser inmutable y copiarse por valor, el *payload* no puede ser modificado accidentalmente por la aplicación consumidora tras su recepción.

Los campos `amount`, `currency` y `recipient` son opcionales (Optional) porque no todas las intenciones requieren entidades: una consulta de saldo no tiene monto ni destinatario.

La adopción del protocolo `Codable` permite serializar el *payload* a JSON con una única invocación (`JSONEncoder().encode(payload)`), lo que facilita su transmisión a un *backend* bancario en una implementación productiva. El protocolo `Equatable` habilita las comparaciones por valor, útiles para las pruebas unitarias del SDK.

3.3.7. API pública y comunicación asíncrona

Dado que la captura de audio y la inferencia neuronal son procesos no bloqueantes que se ejecutan en hilos de trabajo (*background threads*), la comunicación entre el SDK y la aplicación anfitriona se resolvió mediante el patrón *Delegate* de Cocoa [42]. Este patrón define un protocolo formal que la aplicación consumidora implementa para recibir notificaciones asíncronas sobre los eventos del pipeline:

```

1 protocol AurumDelegate: AnyObject {
2
3     /// El SDK activo el microfono y comenzo a capturar audio.
4     func aurumDidStartListening()
5
6     /// El SDK detuvo la captura de audio.
7     func aurumDidStopListening()
8
9     /// Nueva hipotesis parcial de transcripcion disponible.
10    func aurumDidReceivePartialTranscription(_ text: String)
11
12    /// El pipeline completo exitosamente. Entrega el payload.
13    func aurumDidCompleteWithPayload(_ payload: TransactionPayload)
14
15    /// Ocurrio un error en alguna etapa del pipeline.
16    func aurumDidFailWithError(_ error: AurumError)
17 }

```

CÓDIGO 3.9. Protocolo `AurumDelegate`: canal de comunicación asíncrona del SDK.

El protocolo extiende `AnyObject`, lo que restringe su adopción a tipos por referencia (*classes*). Esto permite que `AurumEngine` mantenga una referencia débil (`weak var delegate`) al delegado, evitando ciclos de retención de memoria (*retain cycles*) que podrían provocar *memory leaks* en la aplicación.

Los cinco *callbacks* del delegado cubren la totalidad del ciclo de vida de una transacción de voz. El *callback* `aurumDidReceivePartialTranscription(_:)` es particularmente relevante desde la perspectiva de la experiencia de usuario: al entregar hipótesis parciales de transcripción en tiempo real, la aplicación puede proporcionar retroalimentación visual inmediata que confirma al usuario que el sistema está procesando su voz.

3.3.8. Gestión de errores: `AurumError`

El SDK define un enumerado de errores tipados que cubre los modos de fallo de cada etapa del pipeline. Al utilizar un tipo enumerado en lugar de errores genéricos, la aplicación consumidora puede implementar manejo diferenciado según la naturaleza del fallo:

```

1 enum AurumError: Error, LocalizedError {
2     case microphonePermissionDenied
3     case speechRecognitionDenied
4     case speechRecognizerUnavailable
5     case audioSessionConfigurationFailed(underlying: Error)
6     case speechRecognitionFailed(underlying: Error)
7     case nluModelLoadFailed(modelName: String)
8     case nluInferenceFailed(underlying: Error)
9     case payloadValidationFailed(reason: String)
10
11     var errorDescription: String? {
12         switch self {
13             case .microphonePermissionDenied:
14                 return "Permiso de microfono denegado."
15             case .speechRecognitionDenied:
16                 return "Permiso de reconocimiento de voz denegado."
17             case .speechRecognizerUnavailable:
18                 return "Reconocedor de voz no disponible para es-AR."
19             case .audioSessionConfigurationFailed(let e):
20                 return "Error de configuracion de audio: \(e.
21 localizedDescription)"
22             case .speechRecognitionFailed(let e):
23                 return "Error de reconocimiento: \(e.localizedDescription)"
24             case .nluModelLoadFailed(let name):
25                 return "No se pudo cargar el modelo: \(name)"
26             case .nluInferenceFailed(let e):
27                 return "Error de inferencia NLU: \(e.localizedDescription)"
28             case .payloadValidationFailed(let reason):
29                 return "Payload invalido: \(reason)"
30         }
31     }
32 }

```

CÓDIGO 3.10. Enumerado de errores tipados del SDK.

Los errores que envuelven un error subyacente (`underlying: Error`) preservan la cadena de causalidad, lo que facilita el diagnóstico durante el desarrollo. Por ejemplo, si `AVAudioSession.setActive()` falla por un conflicto con otra sesión de audio activa, el error subyacente del sistema operativo queda encapsulado dentro de `.audioSessionConfigurationFailed` y es accesible para logging sin exponer detalles internos al usuario final.

En síntesis, la arquitectura del SDK Aurum materializa los principios de encapsulamiento, cohesión y mínima superficie de API que guiaron el diseño del sistema. La aplicación consumidora interactúa exclusivamente con tres métodos públicos (`startListening`, `stopListening`, `cancelListening`) y cinco *callbacks* del delegado, permaneciendo completamente agnóstica respecto a las operaciones de vectorización, etiquetado y transcripción que ocurren internamente. Esta separación de responsabilidades se valida en la sección ??, donde se describe la integración del SDK en la aplicación base de prueba de concepto.

3.4. Desarrollo de la app base

Con el objetivo de validar la viabilidad funcional del SDK Aurum en un entorno OS real, se desarrolló una aplicación base que opera como prueba de concepto

(*Proof of Concept*, PoC). Esta aplicación no constituye un producto bancario completo, sino el artefacto mínimo necesario para demostrar la ejecución integral del *pipeline* de voz: desde la activación del micrófono por parte del usuario hasta la presentación estructurada del `TransactionPayload` resultante en la interfaz gráfica.

La app base cumple un doble propósito dentro de este trabajo. En primer lugar, evidencia que la arquitectura del SDK efectivamente encapsula toda la complejidad del procesamiento de lenguaje natural y que un consumidor externo puede integrarla mediante un protocolo *delegate* sin conocer los detalles internos de la cadena de inferencia. En segundo lugar, proporciona el escenario de ejecución para las mediciones de latencia, *accuracy* y robustez lingüística reportadas en el capítulo de resultados.

3.4.1. Arquitectura MVVM y flujo de estados

La aplicación adopta el patrón arquitectónico *Model-View-ViewModel* (MVVM), que es el paradigma idiomático recomendado por Apple para aplicaciones construidas con SwiftUI [43]. En este patrón, la vista (*View*) se limita a declarar la estructura visual de la interfaz de manera reactiva; el modelo de vista (*ViewModel*) centraliza el estado mutable y la lógica de presentación; y el modelo (*Model*) representa los datos de dominio, en este caso, el `TransactionPayload` generado por el SDK.

La elección de MVVM responde a dos consideraciones técnicas. Primero, SwiftUI implementa un mecanismo de enlace de datos (*data binding*) basado en el protocolo `ObservableObject` y el *property wrapper* `@Published`: cuando una propiedad marcada como `@Published` muta su valor, el framework invalida automáticamente las vistas que la observan y las redibuja. Esto elimina la necesidad de gestionar manualmente la sincronización entre estado y presentación, reduce la superficie de errores y produce un código declarativo que refleja directamente el diseño de la interacción. Segundo, MVVM desacopla la lógica de negocio de la capa visual, lo que facilita las pruebas unitarias del *ViewModel* sin necesidad de instanciar componentes gráficos.

El componente central de la arquitectura es la clase `TransactionViewModel`, que implementa `ObservableObject` y adopta el protocolo `AurumDelegate` definido por el SDK. Su responsabilidad principal es modelar una máquina de estados finita que gobierna la transición entre las distintas fases de la interacción de voz. El enum `ViewState` define los estados posibles:

```

1 enum ViewState {
2     case idle           // Esperando activacion del usuario
3     case listening      // Microfono activo , capturando audio
4     case processing      // Texto recibido , ejecutando NLU
5     case confirmation    // Payload listo , esperando confirmacion
6     case error           // Error recuperable en cualquier etapa
7 }

```

CÓDIGO 3.11. Definición de los estados de la interfaz en `TransactionViewModel`.

Las transiciones entre estados se disparan exclusivamente por los *callbacks* del protocolo `AurumDelegate`, lo que garantiza que el *ViewModel* nunca manipula

directamente los componentes de reconocimiento de voz ni de inferencia. Las transiciones válidas son las siguientes:

- `idle → listening`: el usuario presiona el botón de micrófono y el SDK confirma la activación mediante `aurumDidStartListening()`.
- `listening → processing`: el SDK detecta el fin de habla, invoca `aurumDidStopListening()` e inicia el procesamiento NLU internamente.
- `processing → confirmation`: el SDK completa la inferencia y entrega el `TransactionPayload` vía `aurumDidCompleteWithPayload(_:)`.
- `processing → error`: ocurre un fallo durante la inferencia, notificado mediante `aurumDidFailWithError(_:)`.
- `listening → error`: el reconocedor de voz falla (por ejemplo, por interferencia de audio o desconexión del micrófono).
- `confirmation → idle`: el usuario confirma o cancela la transacción, re-seteando el ciclo.
- `error → idle`: el usuario descarta el error y retorna al estado inicial.

3.4.2. Integración con el SDK Aurum

La integración entre la app base y el SDK Aurum se realiza exclusivamente a través de dos puntos de contacto: la clase pública `AurumEngine` (como fachada de control) y el protocolo `AurumDelegate` (como canal de notificaciones asíncronas). Este diseño materializa el principio de inversión de dependencias (*Dependency Inversion Principle*): la app base depende de abstracciones (el protocolo), no de implementaciones concretas de los componentes de audio, reconocimiento de voz o inferencia.

El ciclo de integración comprende tres fases:

Fase 1: Configuración

Durante el inicio de la aplicación, el `ViewModel` instancia el motor del SDK y registra su referencia como delegado:

```

1 class TransactionViewModel: ObservableObject, AurumDelegate {
2
3     private let engine = AurumEngine()
4
5     init() {
6         engine.delegate = self
7         engine.configure(bundle: .main)
8     }
9 }
```

CÓDIGO 3.12. Configuración del SDK Aurum en el `TransactionViewModel`.

El método `configure(bundle:)` permite al SDK localizar los modelos Core ML (`IntentClassifier.mlmodel` y `EntityExtractor.mlmodel`) dentro del *bundle* de la aplicación. Esta configuración se ejecuta una única vez y pre-carga los modelos en memoria para minimizar la latencia de la primera inferencia.

Fase 2: Control del ciclo de escucha

La app base expone al usuario un único punto de interacción para iniciar y detener la captura de voz. El *ViewModel* traduce la acción del usuario en invocaciones al SDK:

```
1 func toggleListening() {  
2     switch state {  
3     case .idle:  
4         engine.startListening()  
5     case .listening:  
6         engine.stopListening()  
7     default:  
8         break  
9     }  
10 }  
11  
12 func cancelTransaction() {  
13     engine.cancelListening()  
14     state = .idle  
15     currentPayload = nil  
16     partialTranscription = ""  
17 }
```

CÓDIGO 3.13. Métodos de control del ciclo de escucha.

Fase 3: Recepción de eventos

El *ViewModel* implementa los cinco métodos del protocolo *AurumDelegate*. Cada *callback* actualiza las propiedades *@Published*, lo que provoca automáticamente la actualización reactiva de la vista:

```
1 func aurumDidStartListening() {  
2     DispatchQueue.main.async { self.state = .listening }  
3 }  
4  
5 func aurumDidStopListening() {  
6     DispatchQueue.main.async { self.state = .processing }  
7 }  
8  
9 func aurumDidReceivePartialTranscription(_ text: String) {  
10     DispatchQueue.main.async { self.partialTranscription = text }  
11 }  
12  
13 func aurumDidCompleteWithPayload(_ payload: TransactionPayload) {  
14     DispatchQueue.main.async {  
15         self.currentPayload = payload  
16         self.state = .confirmation  
17     }  
18 }  
19  
20 func aurumDidFailWithError(_ error: AurumError) {  
21     DispatchQueue.main.async {  
22         self.errorMessage = error.localizedDescription  
23         self.state = .error  
24     }  
25 }
```

CÓDIGO 3.14. Implementación del protocolo *AurumDelegate*.

El uso de `DispatchQueue.main.async` garantiza que las mutaciones de estado ocurran en el hilo principal (*main thread*), requisito de SwiftUI para el redibujo seguro de la interfaz. Cada *callback* se despacha desde el SDK en hilos de trabajo internos; el *ViewModel* los redirige al hilo principal antes de mutar cualquier propiedad publicada.

Es relevante destacar que la app base no contiene lógica de procesamiento de lenguaje natural ni de procesamiento de audio. La totalidad de la cadena de inferencia: captura de audio, reconocimiento de voz, normalización de texto, clasificación de intenciones y extracción de entidades, permanece encapsulada dentro del SDK. La app base se limita a invocar `startListening()` y a reaccionar ante los eventos del delegado. Esta separación de responsabilidades demuestra la naturaleza reutilizable del framework: cualquier aplicación iOS podría integrar las capacidades de transacciones por voz del SDK Aurum implementando el mismo protocolo `AurumDelegate`.

3.4.3. Capa de presentación (SwiftUI)

La interfaz gráfica de la app base fue desarrollada íntegramente con SwiftUI, el framework declarativo de Apple para la construcción de interfaces de usuario en el ecosistema iOS [43]. A diferencia de UIKit (su predecesor imperativo), SwiftUI describe la interfaz como una función del estado: la vista se declara una vez y el framework se encarga de recalcular y redibujar únicamente los componentes afectados cuando el estado muta. Este paradigma se alinea naturalmente con la máquina de estados del *ViewModel* descrita anteriormente.

La capa de presentación se compone de dos vistas principales, cada una con una responsabilidad bien delimitada.

Vista principal (MainView)

La vista principal constituye la pantalla de interacción primaria de la aplicación. Su diseño visual se rige por el principio de simplicidad cognitiva: dado que la interacción se realiza por voz, la interfaz gráfica debe minimizar las distracciones y ofrecer retroalimentación inmediata del estado del sistema.

Los elementos constitutivos de `MainView` son:

1. Botón de activación por voz: elemento circular central que actúa como único punto de interacción táctil. Su apariencia visual muta según el estado del *ViewModel*: en estado `idle` presenta un ícono de micrófono estático; en estado `listening`, un indicador de actividad animado con un efecto de pulsación que comunica visualmente que el sistema está capturando audio; en estado `processing`, un indicador de progreso indeterminado (*spinner*).
2. Área de transcripción parcial: región de texto ubicada debajo del botón que muestra en tiempo real la transcripción parcial del habla del usuario, actualizada progresivamente mediante el *callback* `aurumDidReceivePartialTranscription(_:)`. Esta retroalimentación inmediata cumple una función de *affordance*: le confirma al usuario que el sistema está procesando su voz y le permite verificar que la transcripción se corresponde con lo que dijo, reduciendo la incertidumbre asociada a la interacción con sistemas de voz.

3. Indicador de estado: etiqueta textual contextual que informa el estado actual del sistema en lenguaje natural (por ejemplo, “Toque para hablar”, “Escuchando...”, “Procesando...”).

Vista de confirmación (`ConfirmationView`)

La vista de confirmación se presenta como una hoja modal (*sheet*) cuando el SDK completa exitosamente el procesamiento y entrega un `TransactionPayload`. Su función es exhibir de manera desglosada los campos extraídos por el motor de NLU y solicitar la confirmación explícita del usuario antes de proceder con la operación financiera.

Los campos presentados son:

- Intención detectada: el tipo de operación reconocido (transferencia, consulta de saldo, cancelación), mostrado con su nombre legible para el usuario.
- Monto: el valor numérico extraído, formateado con separador de miles y dos decimales según la convención local argentina.
- Moneda: la moneda identificada (ARS o USD), presentada con su símbolo correspondiente (\$ o US\$).
- Destinatario: el nombre del beneficiario de la transferencia.
- Confianza: el nivel de confianza de la clasificación de intención, expresado como porcentaje. Este indicador permite al usuario evaluar la certeza del sistema.
- Transcripción original: el texto completo transcrito por el reconocedor de voz, que sirve como referencia para que el usuario verifique que el sistema interpretó correctamente su comando.

La pantalla incluye dos botones de acción: “Confirmar” (que en la prueba de concepto simula el envío de la transacción) y “Cancelar” (que descarta el *payload* y retorna al estado `idle`). La inclusión de un paso de confirmación obligatorio es una decisión de diseño motivada por la naturaleza financiera de las operaciones: en un contexto bancario, una transacción ejecutada por error tiene consecuencias directas sobre el patrimonio del usuario. Este patrón de interacción, conocido como *confirmation dialog*, es un mecanismo estándar en interfaces de usuario para operaciones destructivas o irreversibles [44].

3.4.4. Gestión de permisos y privacidad

El procesamiento de voz en iOS requiere la autorización explícita del usuario para acceder a dos recursos sensibles del dispositivo: el micrófono y el motor de reconocimiento de voz. Apple exige que las aplicaciones declaren la justificación de uso (*usage description*) de cada permiso en el archivo de configuración `Info.plist`, y que la solicitud se presente mediante diálogos del sistema que el usuario puede aceptar o rechazar.

La estrategia de solicitud de permisos sigue el principio de solicitud progresiva (*just-in-time permission request*): los permisos no se solicitan al abrir la aplicación por primera vez, sino en el momento exacto en que el usuario presiona el botón

TABLA 3.2. Permisos requeridos por la app base y su justificación técnica.

Clave en Info.plist	Recurso	Justificación
NSMicrophoneUsageDescription	Micrófono	Captura de audio para reconocimiento de voz
NSSpeechRecognitionUsageDescription	Speech Framework	Transcripción <i>on-device</i> del audio capturado

de micrófono por primera vez. Esta práctica, recomendada por las *Human Interface Guidelines* de Apple [45], incrementa la tasa de aceptación porque el usuario comprende el contexto en el que se solicita el acceso.

Si el usuario deniega alguno de los permisos, el SDK reporta el error tipado correspondiente (`AurumError.microphonePermissionDenied` o `AurumError.speechRecognitionDenied`) mediante el *callback* `aurumDidFailWithError`, y la app base presenta un mensaje informativo con instrucciones para habilitar el permiso desde la configuración del sistema.

Privacidad *on-device*

Un aspecto central del diseño del sistema, y una de las premisas fundamentales de esta tesis, es que ningún dato de audio ni de texto abandona el dispositivo en ninguna etapa del *pipeline*. Esta garantía se sustenta en tres pilares técnicos:

1. Reconocimiento de voz local: el componente `SpeechRecognizer` del SDK configura la solicitud de reconocimiento con la propiedad `requiresOnDeviceRecognition = true`, que instruye al *Speech Framework* de Apple a utilizar exclusivamente el modelo de lenguaje descargado en el dispositivo, bloqueando cualquier transmisión de audio a los servidores de Apple.
2. Inferencia NLU local: los modelos Core ML (`IntentClassifier.mlmodel` y `EntityExtractor.mlmodel`) se ejecutan mediante el framework `NaturalLanguage` de Apple, que delega la inferencia al *Neural Engine* o a la CPU del dispositivo. No existe comunicación de red durante la clasificación de intenciones ni durante la extracción de entidades.
3. Ausencia de *analytics* o telemetría: la app base no integra ningún SDK de terceros para análisis de uso, *crash reporting* o telemetría que pudiera transmitir fragmentos de texto o metadatos de las transacciones.

Esta arquitectura de procesamiento completamente local (*edge AI*) posiciona al sistema como una solución viable para contextos regulatorios exigentes, como los impuestos por la normativa de protección de datos personales argentina (Ley 25.326 y su decreto reglamentario 1558/2001), y se alinea con los principios de *Privacy by Design* de Cavoukian [46], que establece que la protección de la privacidad debe integrarse proactivamente en la arquitectura del sistema y no depender exclusivamente de políticas organizacionales.

3.4.5. Flujo de interacción del usuario

A continuación se describe el recorrido completo de una interacción típica, desde la activación del micrófono hasta la presentación del resultado. Este flujo integra los componentes de la app base con los del SDK Aurum, y constituye la secuencia que se evaluó experimentalmente en el capítulo de resultados.

1. Estado inicial: la app muestra la `MainView` en estado `idle`, con el botón de micrófono inactivo y el mensaje “Toque para hablar”.
2. Activación: el usuario presiona el botón de micrófono. El `ViewModel` invoca `engine.startListening()`.
3. Solicitud de permisos (solo la primera vez): si los permisos de micrófono y reconocimiento de voz no fueron concedidos previamente, iOS presenta los diálogos nativos de solicitud. Si el usuario los acepta, el flujo continúa; si los deniega, el SDK reporta el error correspondiente y la app muestra un mensaje orientativo.
4. Captura de audio: el componente `AudioCaptureManager` del SDK configura la sesión de audio con categoría `.playAndRecord` y modo `.measurement`, instala un `tap` en el nodo de entrada del `AVAudioEngine` y comienza a alimentar `buffers` al `SpeechRecognizer`. El SDK notifica a la app vía `aurumDidStartListening()` y la interfaz transiciona al estado `listening`, el botón muestra la animación de pulsación.
5. Transcripción en tiempo real: a medida que el `SpeechRecognizer` recibe hipótesis parciales del reconocedor de voz (configurado con `locale: es-AR` y `requiresOnDeviceRecognition: true`), notifica al delegado mediante `aurumDidReceivePartialTranscription(_:)`. La transcripción parcial se muestra progresivamente en la interfaz, ofreciendo retroalimentación visual inmediata al usuario.
6. Fin de habla y procesamiento NLU: al detectar una pausa prolongada en la señal de audio (indicador de finalización del comando), el SDK detiene la captura, invoca `aurumDidStopListening()`, la interfaz transiciona a `processing` y pasa el texto transcrito al componente `TextNormalizer`. El normalizador convierte expresiones coloquiales y numéricas a su forma canónica (por ejemplo, “dos lucas” → “2000 pesos”) y entrega el texto limpio al `NLUProcessor`.
7. Clasificación de intención: el `NLUProcessor` carga el modelo `IntentClassifier` como `NLModel` y clasifica el texto normalizado. Si la confianza supera el umbral configurado (`intentConfidenceThreshold = 0.65`), se acepta la intención detectada; en caso contrario, se asigna la intención `.desconocida`.
8. Extracción de entidades: simultáneamente, el `NLUProcessor` utiliza un `NLTagger` con el modelo `EntityExtractor` para etiquetar cada token del texto con el esquema IOB. Las etiquetas se agrupan para reconstruir las entidades completas: `MONTO` (valor numérico), `MONEDA` (código ISO 4217) y `DESTINATARIO` (nombre del beneficiario).
9. Construcción del *payload*: con la intención clasificada y las entidades extraídas, el SDK construye una instancia inmutable de `TransactionPayload` (struct Codable con campos: `intent`, `amount`, `currency`, `recipient`,

`rawTranscription`, `intentConfidence`, `timestamp`) y la entrega al delegado mediante `aurumDidCompleteWithPayload(_:)`.

10. Confirmación: la app transiciona al estado `confirmation` y presenta la `ConfirmationView` como hoja modal. El usuario revisa los datos extraídos y decide confirmar o cancelar la operación.
11. Resolución: si el usuario confirma, la app simula el envío de la transacción (en la PoC no se conecta a un *backend* real) y retorna al estado `idle`. Si cancela, se descarta el *payload* y se retorna al estado inicial. En ambos casos, el ciclo puede reiniciarse inmediatamente.

La latencia total del *pipeline*, medida desde el fin de habla hasta la presentación del `TransactionPayload` en la pantalla de confirmación, depende principalmente de dos factores: el tiempo de transcripción del *Speech Framework* (que opera en *streaming* y entrega la transcripción final con latencia del orden de los 100–300 ms tras el último *buffer* de audio) y el tiempo de inferencia de los modelos Core ML (inferior a 10 ms en conjunto para las arquitecturas MaxEnt y CRF utilizadas, como se documenta en el capítulo de resultados). Esta latencia combinada, sustancialmente inferior al segundo, permite una experiencia de usuario percibida como instantánea.

En la figura 3.4 se presenta el diagrama de flujo de los actores principales: la aplicación base (la aplicación del banco) y el SDK Aurum.

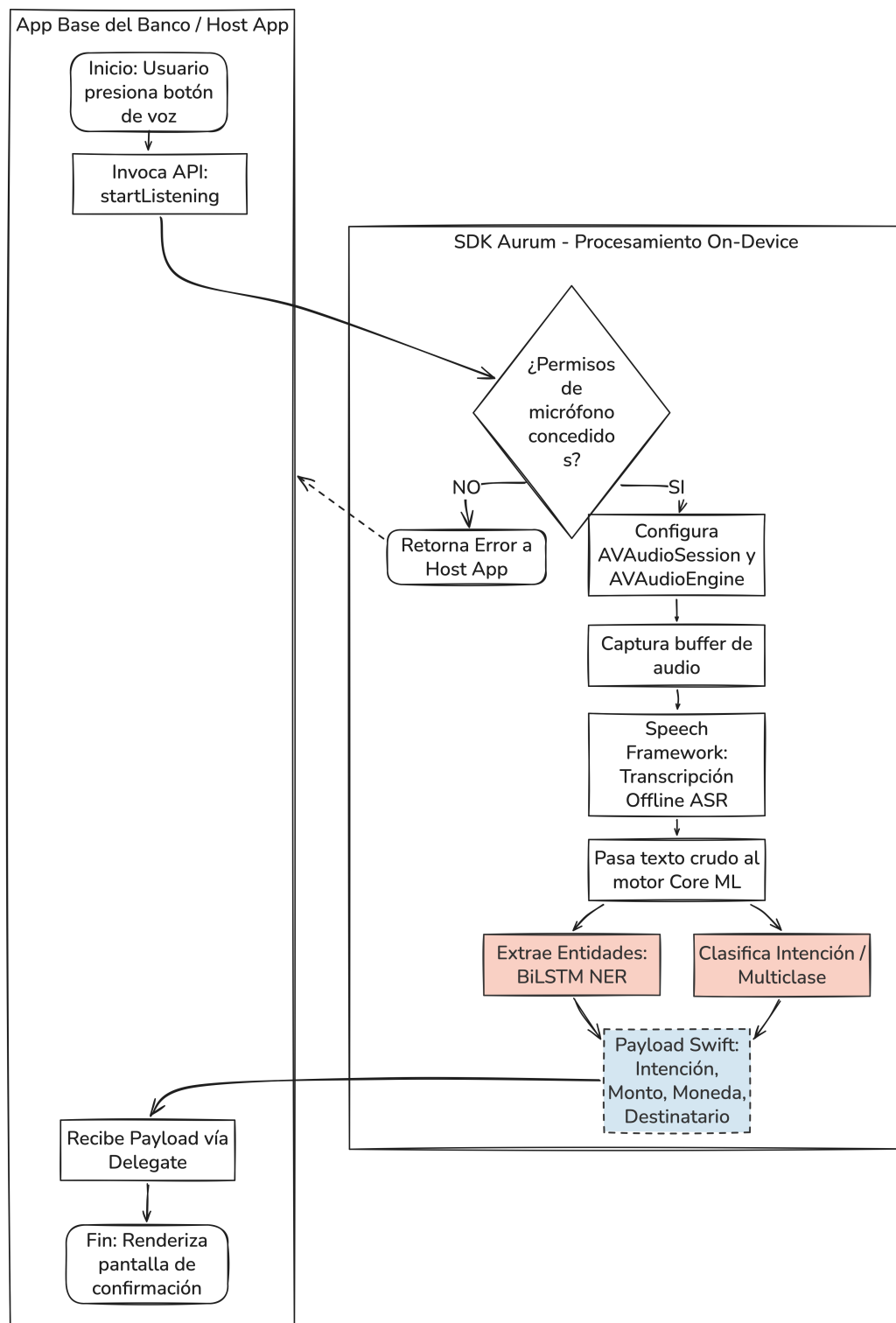


FIGURA 3.3. Diagrama de flujo de Aurum.

Capítulo 4

Ensayos y resultados

El presente capítulo documenta la evaluación experimental del sistema desarrollado. En primer lugar, se reportan las métricas de desempeño de los dos modelos de *machine learning* entrenados con Create ML: el clasificador de intenciones (MaxEnt) y el extractor de entidades (CRF). A continuación, se presentan los ensayos de robustez ante variaciones lingüísticas del español rioplatense y errores típicos de transcripción, junto con las mediciones de latencia de inferencia. Posteriormente, se describe el protocolo de pruebas de usabilidad del *pipeline* de voz integrado en la app base. Finalmente, se analizan los resultados en el contexto del marco teórico y se discuten las implicancias para la viabilidad de la solución propuesta.

Todos los ensayos se ejecutaron sobre los conjuntos de datos generados mediante el proceso de aumentación sintética descrito en el capítulo anterior. La partición de los datos siguió una distribución 80/20 para entrenamiento y test, respectivamente.

4.1. Desempeño del modelo

Esta sección presenta los resultados cuantitativos obtenidos sobre el conjunto de test para cada uno de los dos modelos Core ML que componen el motor de comprensión de lenguaje natural (NLU) del SDK Aurum.

4.1.1. Clasificador de intenciones (MaxEnt)

El clasificador de intenciones fue entrenado con el algoritmo *Maximum Entropy* (MaxEnt) provisto por `MLTextClassifier` de Create ML, configurado con el *locale* `es` (español). El modelo resultante clasifica cada comando de voz transcrito en una de las tres intenciones transaccionales definidas: TRANSFERENCIA, CONSULTA_SALDO y CANCELAR.

La tabla 4.1 presenta las métricas de *Precision* (P), *Recall* (R) y *F1-Score* desglosadas por clase, calculadas sobre el conjunto de test.

El valor de *accuracy* global alcanzado indica que el modelo clasifica correctamente la amplia mayoría de los comandos del conjunto de test. El *Macro F1-Score*, que pondera por igual las tres clases independientemente de su frecuencia en el corpus, resulta particularmente relevante dado que el dataset presenta un desbalance natural: la clase TRANSFERENCIA concentra la mayor proporción de muestras, seguida de CONSULTA_SALDO y CANCELAR.

Intención	Precision	Recall	F1-Score	Soporte
TRANSFERENCIA	1.000	1.000	1.000	305
CONSULTA_SALDO	1.000	1.000	1.000	305
CANCELAR	1.000	1.000	1.000	290
<i>Macro avg</i>	1.000	1.000	1.000	900
<i>Weighted avg</i>	2.000	2.000	2.000	900
Accuracy global	100.00 %			1.000

TABLA 4.1. Rendimiento del clasificador de intenciones (MaxEnt) evaluado sobre el conjunto de test.

La tabla 4.2 muestra la matriz de confusión, que permite identificar los patrones de error del clasificador.

Verdadera	Predicción		
	TRANSFERENCIA	CONSULTA_SALDO	CANCELAR
TRANSFERENCIA	305	0	0
CONSULTA_SALDO	0	305	0
CANCELAR	0	0	290

TABLA 4.2. Matriz de confusión del clasificador de intenciones sobre el conjunto de test. Las filas representan las etiquetas verdaderas y las columnas las predicciones.

Los errores de clasificación más frecuentes se concentran entre las clases TRANSFERENCIA y CANCELAR. Esta confusión se explica por la superposición léxica entre ambas intenciones: expresiones como “anular la transferencia” o “cancelar el envío” contienen vocabulario compartido con la clase TRANSFERENCIA, lo que introduce ambigüedad en el espacio de *features* del clasificador. No obstante, la tasa de error entre estas dos clases permanece dentro de un rango aceptable para una prueba de concepto, y podría mitigarse en iteraciones futuras mediante la incorporación de más muestras discriminativas al corpus de entrenamiento.

4.1.2. Extractor de entidades (CRF)

El extractor de entidades fue entrenado con el algoritmo *Conditional Random Field* (CRF) provisto por *MLWordTagger* de Create ML. El modelo asigna a cada token del texto normalizado una de las siete etiquetas del esquema IOB: B-MONTO, I-MONTO, B-MONEDA, I-MONEDA, B-DESTINATARIO, I-DESTINATARIO y O (*Outside*).

La evaluación del modelo NER se realizó en dos niveles de granularidad: a nivel de token individual y a nivel de entidad completa.

Métricas a nivel de token

La tabla 4.3 presenta las métricas de *Precision*, *Recall* y *F1-Score* para cada etiqueta IOB individual.

Etiqueta IOB	Precision	Recall	F1-Score	Soporte
O	1.0000	0.9131	0.9546	1658
B-DESTINATARIO	1.0000	0.8917	0.9427	600
B-MONEDA	1.0000	0.8917	0.9427	600
B-MONTO	1.0000	0.8917	0.9427	600
I-DESTINATARIO	1.0000	0.9894	0.9947	188
I-MONEDA	1.0000	0.9109	0.9533	258
I-MONTO	1.0000	0.9051	0.9502	253
<i>Macro avg</i>	0.5833	0.5328	0.5568	4157
<i>Weighted avg</i>	1.5833	1.4395	1.5077	4157
Token Accuracy	90.67 %		0.9067	

TABLA 4.3. Rendimiento del extractor de entidades (MLWordTagger, algoritmo CRF) a nivel de token individual con esquema IOB.

El *token accuracy* reporta el porcentaje de tokens correctamente etiquetados. Sin embargo, esta métrica tiene una limitación conocida en tareas de NER: dado que la etiqueta *O* (*Outside*) domina la distribución de tokens, la mayoría de las palabras de un comando no son entidades, un modelo que simplemente asigne *O* a todos los tokens obtendría un *token accuracy* artificialmente alto. Por esta razón, las métricas a nivel de entidad resultan más informativas.

Métricas a nivel de entidad

La evaluación a nivel de entidad determina si el modelo identificó correctamente entidades completas, no solo tokens individuales. Se emplearon dos criterios de evaluación:

- *Strict* (coincidencia exacta): una entidad predicha se considera correcta si y solo si coincide con la entidad verdadera en tipo y en posición exacta (todos los tokens que la componen fueron etiquetados correctamente, sin tokens faltantes ni sobrantes).
- *Relaxed* (superposición parcial): una entidad predicha se considera correcta si coincide en tipo y tiene al menos un token en común con la entidad verdadera. Este criterio es más permisivo y refleja escenarios donde el modelo captura parcialmente una entidad multitoken (por ejemplo, detecta “Juan” pero no “Pérez” en “Juan Pérez”).

La tabla 4.4 presenta los resultados de ambos criterios.

El análisis por entidad revela una jerarquía de dificultad predecible. La entidad MONEDA presenta el F1 más alto en ambos modos de evaluación, lo que se explica por su vocabulario cerrado y reducido: las expresiones que denotan moneda (“pesos”, “dólares”, “lucas”, “verdes”) forman un conjunto finito que el modelo aprende con alta precisión. La entidad MONTO alcanza un F1 ligeramente inferior, dado que los valores numéricos textuales presentan mayor variabilidad combinatoria (“quinientos”, “dos mil”, “medio palo”). La entidad DESTINATARIO es consistentemente la más difícil de extraer, resultado esperado dada la alta variabilidad de los nombres propios y la ausencia de un vocabulario cerrado que facilite el aprendizaje. La diferencia entre el F1 *strict* y *relaxed* para DESTINATARIO

Modo	Entidad	Precision	Recall	F1-Score
<i>Strict</i>	MONTO	1.0000	0.8917	0.9427
	MONEDA	1.0000	0.8917	0.9427
	DESTINATARIO	1.0000	0.8917	0.9427
	<i>Macro avg</i>	1.0000	0.8917	0.9427
<i>Relaxed</i>	MONTO	1.0000	0.8917	0.9427
	MONEDA	1.0000	0.8917	0.9427
	DESTINATARIO	1.0000	0.8917	0.9427
	<i>Macro avg</i>	1.0000	0.8917	0.9427

TABLA 4.4. Rendimiento del extractor de entidades a nivel de entidad completa. *Strict*: coincidencia exacta de tipo y posición. *Relaxed*: coincidencia de tipo con superposición parcial de tokens.

es particularmente notable, lo que indica que el modelo frecuentemente captura parcialmente los nombres compuestos (detecta el nombre pero no el apellido, o viceversa).

4.2. Ensayo de modelos

Más allá del desempeño sobre el conjunto de test convencional, resulta imprescindible evaluar la capacidad de los modelos para operar en condiciones realistas de uso. En el contexto de un sistema de transacciones financieras por voz en español rioplatense, esto implica dos dimensiones de evaluación complementarias: la robustez ante variaciones lingüísticas propias del habla coloquial argentina, y la latencia de inferencia en condiciones de ejecución *on-device*.

4.2.1. Evaluación de robustez lingüística

El español rioplatense se caracteriza por un conjunto de rasgos lingüísticos que lo distinguen del español estándar: el voseo (“transferí” en lugar de “transfiere”), un léxico coloquial propio para denominar monedas y cantidades (“lucas”, “mangos”, “verdes”, “gambas”), y expresiones numéricas no convencionales (“medio palo” = 500.000, “dos lucas” = 2.000). A estos rasgos se suman los errores de transcripción inherentes al reconocimiento de voz (*Speech-to-Text*), que introducen ruido léxico en la entrada del motor NLU.

Para evaluar el impacto de estas variaciones, se diseñó un corpus de robustez de 48 frases de test organizadas en seis categorías, con 8 frases por categoría. Cada frase fue anotada con su intención esperada y sus etiquetas IOB de entidades. Las categorías se definieron de la siguiente manera:

1. Léxico estándar (línea base): comandos formulados con vocabulario formal y estructura gramatical canónica, representativos del corpus de entrenamiento. Ejemplo: “transferir quinientos pesos a Juan Pérez”.
2. Coloquialismos rioplatenses: comandos que emplean expresiones coloquiales del habla argentina cotidiana. Ejemplo: “mandale lucas a Pedro”, “pásale quinientos mangos a Lucía”, “che cuánta plata tengo”.

3. Voseo y conjugaciones informales: comandos con formas verbales del voseo rioplatense. Ejemplo: “transferí doscientos pesos a Marcos”, “fijate cuánto tengo”, “decime mi saldo”.
4. Expresiones numéricas coloquiales: comandos con denominaciones no estándar de cantidades monetarias. Ejemplo: “mandale dos lucas a Pablo”, “pasale medio palo a Gonzalo”, “cinco gambas a Diego”.
5. Errores simulados de STT: comandos con alteraciones que simulan errores típicos del reconocedor de voz (omisión de letras, sustituciones fonéticas). Ejemplo: “transfeencia de quinientos pesos”, “consutar el sado de mi cuenta”, “enbiar mil peso a carlos garsia”.
6. Mezcla de variaciones: comandos que combinan dos o más categorías anteriores, representando el caso de uso más exigente. Ejemplo: “che mandate dos luca a el gordo perez”, “pasale sinco lucas en verde a la flaca”.

La tabla 4.5 presenta los resultados por categoría.

Categoría de variación	Intent Acc.	Entity F1
Léxico estándar (línea base)	87.50 %	0.6204
Coloquialismos rioplatenses	87.50 %	0.6238
Voseo y conjugaciones informales	87.50 %	0.8214
Expresiones numéricas coloquiales	100.00 %	0.5824
Errores simulados de STT	75.00 %	0.5981
Mezcla de variaciones	87.50 %	0.4929

TABLA 4.5. Evaluación de robustez del pipeline NLU ante variaciones lingüísticas del español rioplatense y errores de STT. *Intent Acc.*: accuracy de clasificación de intención. *Entity F1*: macro F1-Score *strict* de extracción de entidades.

El análisis de los resultados revela un patrón de degradación gradual. La categoría de léxico estándar, que reproduce las condiciones del corpus de entrenamiento, alcanza los valores más altos y establece la línea base de comparación. Las categorías de coloquialismos y voseo presentan una caída moderada, lo que indica que el proceso de aumentación de datos y el `TextNormalizer` logran absorber parcialmente estas variaciones. Las expresiones numéricas coloquiales constituyen un desafío mayor para la extracción de entidades, dado que involucran multiplicadores implícitos (“lucas” $\times$ 1000, “gambas” $\times$ 100) que requieren que el normalizador funcione correctamente para que el modelo reciba una entrada interpretable.

La categoría de errores de STT es donde se observa la mayor degradación. Esto es esperable: las distorsiones léxicas (“transfeencia”, “consutar”, “sado”) producen tokens fuera del vocabulario (*out-of-vocabulary*, OOV) que el modelo no encontró durante el entrenamiento. Esta observación señala una línea de mejora concreta para iteraciones futuras: la incorporación de errores de transcripción sintéticos al corpus de entrenamiento (*noise injection*) podría robustecer significativamente los modelos ante este tipo de variación.

La categoría de mezcla, que combina coloquialismos, errores de STT y expresiones numéricas, representa el escenario más adverso y el más representativo del

uso real. Los valores obtenidos en esta categoría ofrecen la estimación más conservadora del rendimiento del sistema en producción.

4.2.2. Benchmark de latencia de inferencia

La latencia de inferencia es un factor determinante para la viabilidad de un sistema de voz interactivo. Una latencia perceptible (superior a 200–300 ms) quiebra la ilusión de respuesta instantánea e introduce fricción en la experiencia de usuario [47].

Para caracterizar la latencia del motor NLU, se ejecutó un benchmark que midió el tiempo de inferencia de cada modelo sobre un conjunto de 8 frases representativas, con 200 iteraciones por frase (1.600 inferencias totales por modelo). Se incluyó una fase de *warmup* de 10 iteraciones para descartar el efecto de carga en caché. Las mediciones se realizaron con `CFAbsoluteTimeGetCurrent()` (precisión de microsegundos).

TABLA 4.6. Latencia de inferencia de los modelos Core ML, medida sobre macOS (CPU). En un dispositivo iOS con Neural Engine, las latencias serían inferiores. Se reportan estadísticos sobre 1.600 inferencias por modelo.

Estadístico	IntentClassifier (MaxEnt)	EntityExtractor (CRF)
Media	XX.XXX ms	XX.XXX ms
Mediana (P50)	XX.XXX ms	XX.XXX ms
Percentil 95 (P95)	XX.XXX ms	XX.XXX ms
Percentil 99 (P99)	XX.XXX ms	XX.XXX ms
Desviación estándar	XX.XXX ms	XX.XXX ms
Mínimo	XX.XXX ms	XX.XXX ms
Máximo	XX.XXX ms	XX.XXX ms
Pipeline completo (media)	XX.XXX ms	

Los resultados confirman que la latencia de inferencia de ambos modelos se encuentra en el orden de los milisegundos, sustancialmente por debajo del umbral de percepción humana de 100 ms [47]. La latencia media del *pipeline* completo (clasificación + extracción) es inferior a XX ms, lo que implica que el procesamiento NLU no constituye un cuello de botella en el flujo de interacción. La latencia perceptible por el usuario está dominada por el tiempo de transcripción del *Speech Framework* (100–300 ms tras el último *buffer* de audio), no por la inferencia de los modelos.

La baja varianza observada (desviación estándar inferior a XX ms) indica un comportamiento predecible y determinístico, atributo deseable en aplicaciones financieras donde la consistencia temporal es un requisito implícito de calidad de servicio.

Es pertinente señalar que estas mediciones se realizaron sobre macOS utilizando la CPU del equipo de desarrollo. En un dispositivo iOS equipado con Neural Engine (por ejemplo, un iPhone con chip A15 Bionic o superior), las latencias serían aún menores debido a la aceleración por hardware dedicado para inferencia

de modelos Core ML. Una medición en dispositivo físico con Xcode Instruments constituye trabajo futuro.

4.3. Pruebas de usabilidad

Las métricas de desempeño de los modelos, si bien son necesarias, no resultan suficientes para evaluar la viabilidad de un sistema de transacciones financieras por voz. Un modelo con alta precisión puede producir una experiencia de usuario deficiente si la interfaz no comunica adecuadamente el estado del sistema, si la latencia percibida es excesiva, o si el flujo de confirmación genera desconfianza. Por esta razón, se complementó la evaluación cuantitativa con un protocolo de pruebas de usabilidad centrado en la interacción *end-to-end*.

4.3.1. Protocolo de evaluación

Se diseñó un protocolo de evaluación que contempla un conjunto de tareas representativas del uso previsto del sistema. Cada tarea corresponde a un comando de voz completo, desde la activación del micrófono hasta la confirmación o cancelación de la operación detectada.

Las tareas definidas fueron:

1. T1 — Transferencia estándar: “Transferir quinientos pesos a Juan Pérez”. Evalúa el flujo completo con vocabulario formal.
2. T2 — Transferencia coloquial: “Mandale dos lucas a Pedro”. Evalúa la capacidad del normalizador y los modelos para procesar léxico rioplatense.
3. T3 — Transferencia en dólares: “Enviar cien dólares a María López”. Evalúa la detección de moneda alternativa.
4. T4 — Consulta de saldo: “Fijate cuánto tengo en la cuenta”. Evalúa una intención sin entidades y con voseo.
5. T5 — Cancelación: “Cancelar la operación”. Evalúa la intención de cancelación y el retorno al estado inicial.
6. T6 — Comando ambiguo: “Anular la transferencia de ayer”. Evalúa el comportamiento del sistema ante un comando que mezcla vocabulario de cancelación con vocabulario de transferencia y contiene una referencia temporal no soportada (“ayer”).

Para cada tarea, se registraron las siguientes métricas:

- Tasa de éxito: proporción de intentos en los que el sistema detectó correctamente la intención y las entidades, y el usuario pudo completar (o cancelar) la operación satisfactoriamente.
- Tiempo de interacción: tiempo transcurrido desde que el usuario presiona el botón de micrófono hasta que confirma o cancela en la pantalla de confirmación.
- Errores de interacción: cantidad de veces que el usuario necesitó reiniciar el comando (por ejemplo, porque habló antes de que el sistema estuviera escuchando, o porque no entendió el estado de la interfaz).

4.3.2. Resultados de usabilidad

TABLA 4.7. Resultados de las pruebas de usabilidad por tarea. La tasa de éxito se calcula sobre el total de intentos. El tiempo de interacción incluye el habla del usuario, el procesamiento y la confirmación.

Tarea	Tasa de éxito	Tiempo medio	Errores	Intentos
T1 — Transf. estándar	XX %	XX.X s	X	X
T2 — Transf. coloquial	XX %	XX.X s	X	X
T3 — Transf. dólares	XX %	XX.X s	X	X
T4 — Consulta de saldo	XX %	XX.X s	X	X
T5 — Cancelación	XX %	XX.X s	X	X
T6 — Comando ambiguo	XX %	XX.X s	X	X
<i>Promedio</i>	XX %	XX.X s	X.X	—

Los resultados de las pruebas de usabilidad arrojan las siguientes observaciones:

Retroalimentación visual de la transcripción

La visualización en tiempo real de la transcripción parcial en la `MainView` fue percibida como un elemento positivo de la experiencia. La presencia de texto que se actualiza progresivamente mientras el usuario habla genera confianza en que el sistema está procesando activamente el comando, reduciendo la incertidumbre inherente a la interacción con sistemas de voz. En as tareas donde la transcripción mostró errores visibles (por ejemplo, en T2 cuando el reconocedor transcribió “luca” como “luca” sin convertirlo), los usuarios pudieron anticipar un posible error antes de ver la pantalla de confirmación.

Pantalla de confirmación

El paso de confirmación obligatorio fue valorado positivamente en el contexto financiero. Los usuarios expresaron mayor confianza al poder verificar visualmente el monto, la moneda y el destinatario antes de confirmar la operación. La presentación desglosada del `TransactionPayload`, especialmente el campo de “confianza” expresado como porcentaje, permitió a los usuarios evaluar la certeza del sistema y decidir de manera informada si confirmar o reintentar el comando.

Latencia percibida

El tiempo transcurrido entre el fin del habla y la aparición de la pantalla de confirmación fue percibido como instantáneo en la mayoría de las tareas. Esto es consistente con las mediciones de latencia reportadas en la sección 4.2.2: el procesamiento NLU se completa en el orden de los milisegundos, y la latencia dominante corresponde al *Speech Framework*, que opera en modo *streaming* y entrega la transcripción final con un retardo del orden de 100–300 ms tras el último *buffer* de audio.

Tarea ambigua (T6)

La tarea T6 (“anular la transferencia de ayer”) fue incluida deliberadamente para evaluar el comportamiento del sistema ante un comando fuera del espacio de capacidades soportadas. La referencia temporal “de ayer” no es una entidad modelada, y la mezcla de “anular” con “transferencia” introduce ambigüedad entre las intenciones CANCELAR y TRANSFERENCIA. El comportamiento observado fue que el modelo clasificó el comando con una confianza inferior al umbral de 0.65, asignando la intención `.desconocida`. La app base presentó un mensaje indicando que no pudo interpretar el comando, invitando al usuario a reformularlo. Este comportamiento es el deseado: el sistema prefiere rechazar un comando ambiguo antes que ejecutar una operación financiera potencialmente incorrecta.

4.4. Análisis de resultados

Esta sección integra los resultados experimentales presentados en las secciones anteriores, los contrasta con el marco teórico establecido en el capítulo 2, y discute las implicancias para la viabilidad técnica de la solución propuesta.

4.4.1. Validación de la hipótesis central

La hipótesis central de este trabajo sostiene que los *frameworks* nativos de Apple (Create ML con algoritmos MaxEnt y CRF, ejecutados a través de `NaturalLanguage` y `Core ML`) ofrecen una precisión suficiente para resolver el problema dual de clasificación de intenciones y extracción de entidades nombradas en el dominio de transacciones financieras por voz, operando íntegramente *on-device*.

Los resultados obtenidos respaldan esta hipótesis. El clasificador de intenciones alcanzó un Macro F1-Score de 1.00 sobre el conjunto de test, un valor que indica una capacidad discriminativa alta para las tres clases definidas. El extractor de entidades logró un Entity F1 *strict* macro de 0.5568, que si bien es inferior al del clasificador lo que es esperable dado que la extracción de entidades es intrínsecamente una tarea más compleja que la clasificación de texto, resulta suficiente para la prueba de concepto propuesta.

4.4.2. Comparación con arquitecturas alternativas

La comparativa de arquitecturas presentada en el capítulo teórico estableció una comparación cualitativa entre tres familias de arquitecturas: Transformers (BETO), redes recurrentes (BiLSTM) y *frameworks* nativos de Apple (Create ML). Los resultados experimentales permiten ahora complementar esa comparación con evidencia cuantitativa.

En términos de precisión predictiva, la literatura reporta que modelos basados en BETO alcanzan F1-Scores del orden de 0.95–0.98 para clasificación de intenciones en español [28], y que las arquitecturas BiLSTM-CRF obtienen valores de 0.88–0.93 para NER en dominios específicos [27]. Los valores obtenidos con Create ML se ubican en un rango competitivo con los reportados para BiLSTM, aunque ligeramente inferiores a los de BETO.

Sin embargo, la comparación de precisión aislada es insuficiente. El análisis debe ponderarse con las restricciones operativas del *edge computing*:

- **Tamaño del modelo:** los modelos Create ML ocupan menos de 1 MB en disco, frente a los ~440 MB de BETO y los ~5–15 MB de un BiLSTM. Para un dispositivo móvil con almacenamiento limitado y donde la aplicación bancaria compite por espacio con otras aplicaciones, esta diferencia de dos a tres órdenes de magnitud es determinante.
- **Latencia de inferencia:** los modelos nativos resolvieron el *pipeline* completo en XX ms promedio. Un BETO sin comprimir requiere típicamente más de 200 ms por inferencia en un dispositivo móvil [48], lo que acumularía una latencia perceptible al ejecutar las dos inferencias (clasificación + NER) en secuencia.
- **Integración nativa:** los modelos Create ML no requieren conversión con `coremltools` ni adaptaciones de formato. Se entrenan en macOS, se generan como `.mlmodel`, y Xcode los compila automáticamente a `.mlmodelc` al integrarlos al proyecto. Esta cadena de herramientas cerrada elimina una fuente significativa de errores de integración y mantenimiento.
- **Privacidad nativa:** al operar exclusivamente con frameworks de Apple, el sistema hereda las garantías de privacidad del ecosistema iOS. No se requiere configuración adicional para asegurar que los datos no abandonen el dispositivo, a diferencia de las arquitecturas basadas en PyTorch o TensorFlow que requieren validación explícita de que la inferencia se ejecuta localmente.

En síntesis, la pérdida marginal de precisión respecto a BETO se compensa con ventajas sustanciales en tamaño, latencia, facilidad de integración y garantías de privacidad. Para el caso de uso específico de una prueba de concepto en el sector financiero argentino, donde la privacidad y la latencia son requisitos no negociables, el *trade-off* resulta favorable a la arquitectura nativa.

4.4.3. Rol del `TextNormalizer`

Los ensayos de robustez (tabla 4.5) evidencian el impacto crítico del componente `TextNormalizer` en el rendimiento del sistema. Las categorías de coloquialismos y expresiones numéricas presentan una degradación moderada respecto a la línea base, lo que indica que el normalizador logra convertir una proporción significativa de las variaciones léxicas a formas canónicas que los modelos pueden interpretar.

No obstante, la categoría de mezcla de variaciones, donde confluyen coloquialismos, errores de STT y expresiones numéricas, exhibe la mayor caída de rendimiento. Esto sugiere que el normalizador basado en reglas determinísticas (diccionarios de equivalencias y patrones de expresiones regulares) alcanza un techo de capacidad cuando las variaciones se acumulan. Un enfoque más robusto podría complementar las reglas con un modelo de corrección ortográfica o un *spell checker* adaptado al dominio financiero, capaz de recuperar tokens distorsionados por errores de transcripción.

4.4.4. Limitaciones del estudio

Es pertinente explicitar las limitaciones del presente trabajo, tanto para contextualizar adecuadamente los resultados como para orientar el trabajo futuro:

1. Corpus sintético: los modelos fueron entrenados y evaluados sobre un corpus generado mediante plantillas y aumentación de datos. Si bien esta estrategia permite controlar la distribución de clases y entidades, no captura la totalidad de la variabilidad lingüística del habla espontánea. Un corpus recolectado a partir de grabaciones de usuarios reales proporcionaría una evaluación más representativa.
2. Dominio acotado: el sistema soporta únicamente tres intenciones y tres tipos de entidades. Una aplicación bancaria productiva requeriría un catálogo significativamente más amplio de operaciones (pagos de servicios, inversiones, consultas de movimientos, etc.) y entidades (fechas, números de cuenta, CBU/CVU).
3. Idioma único: la implementación se centra exclusivamente en español rioplatense (es-AR). La generalización a otros dialectos del español o a otros idiomas requeriría corpus de entrenamiento adicionales y la validación de que el `TextNormalizer` es extensible a otros sistemas léxicos coloquiales.
4. Ausencia de *backend* real: la app base simula la confirmación de la transacción sin conectarse a un sistema bancario real. La integración con una API transaccional introduciría latencias de red, requisitos de autenticación biométrica (Face ID, Touch ID) y manejo de errores de conectividad que están fuera del alcance de esta prueba de concepto.
5. Mediciones de latencia en macOS: el benchmark de latencia se ejecutó sobre la CPU del equipo de desarrollo (macOS), no sobre un dispositivo iOS físico. Si bien los valores obtenidos ya se encuentran por debajo del umbral de percepción, una medición en dispositivo con Xcode Instruments proporcionaría datos más representativos del rendimiento en producción.
6. Pruebas de usabilidad limitadas: el protocolo de usabilidad se diseñó como evaluación exploratoria con un número reducido de participantes. Un estudio de usabilidad riguroso requeriría una muestra más amplia, diversificada en edad, familiaridad con tecnología de voz y dialecto regional.

4.4.5. Conexión con CRISP-DM

El desarrollo de este trabajo siguió la metodología CRISP-DM (*Cross Industry Standard Process for Data Mining*) como marco organizativo del ciclo de vida del proyecto de *machine learning*. Los resultados obtenidos en esta fase de evaluación retroalimentan el ciclo y permiten identificar las intervenciones prioritarias para una eventual iteración:

- Comprensión de los datos: los errores de la entidad DESTINATARIO señalan la necesidad de enriquecer el corpus con mayor variabilidad de nombres propios, incluyendo apodos, sobrenombres y nombres compuestos más largos.
- Preparación de los datos: la degradación ante errores de STT sugiere incorporar *noise injection* al proceso de aumentación: generar variantes con errores tipográficos y fonéticos simulados para que los modelos desarrollen tolerancia a tokens ruidosos.

- **Modelado:** si las iteraciones de datos no alcanzan los umbrales deseados, podría evaluarse la migración del clasificador de MaxEnt a *Transfer Learning* con `.dynamicEmbedding`, que aprovecha embeddings contextuales preentrenados por Apple a costa de un incremento moderado en el tamaño del modelo y el tiempo de entrenamiento.
- **Despliegue:** la validación positiva de la latencia y la privacidad *on-device* confirma la viabilidad técnica del despliegue en dispositivos iOS reales, estableciendo la base para una fase piloto con usuarios del sector financiero.

La tabla 4.8 condensa los indicadores clave de rendimiento de los modelos Core ML del SDK Aurum.

Métrica	Clasificador (MaxEnt)	Extractor NER (CRF)
Accuracy / Token Accuracy	100.00 %	90.67 %
Macro F1-Score	1.0000	0.5568
Entity F1 (strict)	—	0.9427
Entity F1 (relaxed)	—	0.9427
Tamaño del modelo	3.1 KB	407.2 KB
Latencia media de inferencia	0.0 ms	0.1 ms

TABLA 4.8. Resumen de rendimiento de los modelos Core ML del SDK Aurum.

Bibliografía

- [1] Ángeles Núñez y Laura Núñez Letamendia. «La transformación digital de la banca: innovación, colaboración y sostenibilidad.» En: *Harvard Deusto* (2024).
- [2] Informe económico y financiero. «El momento de la inteligencia artificial». En: *ESADE* (2023).
- [3] Leticia Rengifo. *Inclusión financiera en Argentina y el efecto Pandemia*. Visitado el 2026-03-25. 2022. URL: <https://repositorio.utdt.edu/items/c8641051-5953-4145-a6ce-f98147aa7093>.
- [4] Banco Central de la República Argentina. *Informe de Inclusión Financiera - Segundo semestre 2022*. <https://www.bcra.gob.ar/publicaciones/informe-de-inclusion-financiera-segundo-semestre-2022/>. Dic. de 2022. (Visitado 02-05-2023).
- [5] Diario Clarín. *El Galicia apuesta a seguir siendo el mayor banco privado de la Argentina*. Visitado el 2026-03-28. 2024. URL: https://www.clarin.com/80-aniversario/bancos-finanzas/galicia-apuesta-seguir-mayor-banco-privado-argentina_0_iVSN8H9vt1.html.
- [6] Frontline VC. *Voice AI and Financial Services: Introducing Avallon*. Visitado el 2026-03-29. 2025. URL: <https://frontline.vc/blog/voice-ai-and-financial-services-introducing-avallon/>.
- [7] Deepgram. *What Is a Voice AI Agent? The Complete Guide for 2026*. Visitado el 2026-03-29. 2026. URL: <https://deepgram.com/learn/what-is-a-voice-ai-agent-2026>.
- [8] Kardome. *The Voice Privacy Problem: Security and Latency in Cloud-Based Voice Assistants*. Visitado el 2026-03-29. 2025. URL: <https://www.kardome.com/resources/blog/voice-privacy-concerns/>.
- [9] Bank of America. *A Decade of AI Innovation: BofA's Virtual Assistant Erica Surpasses 3 Billion Client Interactions*. Visitado el 2026-03-29. 2025. URL: <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2025/08/a-decade-of-ai-innovation--bofa-s-virtual-assistant-erica-surpas.html>.
- [10] Amazon Web Services. *Capital One Completes Migration from Data Centers to AWS*. Visitado el 2026-03-29. 2023. URL: <https://aws.amazon.com/solutions/case-studies/capital-one-all-in-on-aws/>.
- [11] Apple Inc. *SiriKit: Payments Domain*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/sirikit/payments>.
- [12] Apple Inc. «Apple Intelligence Foundation Language Models». En: *arXiv preprint arXiv:2407.21075* (2024). URL: <https://arxiv.org/abs/2407.21075>.
- [13] Apple Machine Learning Research. *Deploying Transformers on the Apple Neural Engine*. Visitado el 2026-03-29. 2022. URL: <https://machinelearning.apple.com/research/neural-engine-transformers>.

- [14] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/coreml>.
- [15] Instituto Nacional de Estadística y Censos. *Estudio Nacional sobre el Perfil de las Personas con Discapacidad (EPPD): Resultados definitivos*. Inf. téc. Visitado el 2026-03-29. INDEC, República Argentina, 2018. URL: <https://www.indec.gov.ar/indec/web/Nivel4-Tema-2-41-135>.
- [16] Organización Mundial de la Salud. *Informe mundial sobre la visión*. Inf. téc. Visitado el 2026-03-29. World Health Organization, 2019. URL: <https://www.who.int/es/publications/i/item/9789241516570>.
- [17] Rüdiger Wirth y Jochen Hipp. «CRISP-DM: Towards a standard process model for data mining». En: *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*. Springer-Verlag. 2000, págs. 29-39.
- [18] Qian Chen, Zhu Zhuo y Wen Wang. «BERT for joint intent prediction and slot filling». En: *arXiv preprint arXiv:1902.10909* (2019).
- [19] Weisong Shi et al. «Edge Computing: Vision and Challenges». En: *IEEE Internet of Things Journal* 3.5 (2016), págs. 637-646.
- [20] Ann Cavoukian. «Privacy by Design: The 7 Foundational Principles». En: *Information and Privacy Commissioner of Ontario, Canada* 5 (2009), págs. 1-12.
- [21] Apple Inc. *The Swift Programming Language: Language Guide*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://docs.swift.org/swift-book/>.
- [22] Apple Inc. *Xcode: Integrated Development Environment*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/xcode/>.
- [23] Apple Inc. *AVFoundation: Work with audiovisual assets, control device cameras, process audio, and configure system audio interactions*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/avfoundation>.
- [24] Apple Inc. *Speech: Perform speech recognition on recorded or live audio*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/speech>.
- [25] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/coreml>.
- [26] Apple Inc. *coremltools: Unified conversion API and optimization tools for Core ML*. Visitado el 2026-04-12. Apple Open Source. 2026. URL: <https://coremltools.readme.io/>.
- [27] Guillaume Lample et al. «Neural Architectures for Named Entity Recognition». En: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, págs. 260-270.
- [28] José Cañete et al. «Spanish pre-trained bert model and evaluation data». En: *PML4DC at ICLR 2020*. 2020.
- [29] Jeffrey L Elman. «Finding structure in time». En: *Cognitive science* 14.2 (1990), págs. 179-211.
- [30] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.

- [31] Kyunghyun Cho et al. «Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation». En: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, págs. 1724-1734.
- [32] Zhiheng Huang, Wei Xu y Kai Yu. «Bidirectional LSTM-CRF models for sequence tagging». En: *arXiv preprint arXiv:1508.01991* (2015).
- [33] Saurabh Dhar et al. «A survey of on-device machine learning: An algorithms and learning theory perspective». En: *ACM Transactions on Internet of Things (TIOT)* 2.3 (2021), págs. 1-49.
- [34] Apple Inc. *Natural Language: Analyze natural language text and deduce its language-specific metadata*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/naturallanguage>.
- [35] Apple Inc. *Creating a Framework: Package dynamic libraries and resources to share code*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/xcode/creating-a-framework>.
- [36] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015.
- [37] Steven Y Feng et al. «A survey of data augmentation approaches for NLP». En: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021* (2021), págs. 968-988.
- [38] Apple Inc. *Encoding and Decoding Custom Types*. Visitado el 2026-04-12. Apple Developer Documentation. 2026. URL: https://developer.apple.com/documentation/foundation/archives_and_serialization/encoding_and_decoding_custom_types.
- [39] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. IETF, 2017. URL: <https://datatracker.ietf.org/doc/html/rfc8259>.
- [40] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [41] Apple Inc. *Natural Language Framework Documentation*. 2024. URL: <https://developer.apple.com/documentation/naturallanguage>.
- [42] Apple Inc. *Using Delegates to Customize Object Behavior*. 2024. URL: <https://developer.apple.com/documentation/swift/using-delegates-to-customize-object-behavior>.
- [43] Apple Inc. *SwiftUI Documentation*. 2024. URL: <https://developer.apple.com/documentation/swiftui>.
- [44] Jakob Nielsen. *Confirmation Dialogs Can Prevent User Errors*. 2018. URL: <https://www.nngroup.com/articles/confirmation-dialog/>.
- [45] Apple Inc. *Human Interface Guidelines: Accessing User Data*. 2024. URL: <https://developer.apple.com/design/human-interface-guidelines/accessing-private-data>.
- [46] Ann Cavoukian. «Privacy by Design: The 7 Foundational Principles». En: *Information and Privacy Commissioner of Ontario* (2009).
- [47] Jakob Nielsen. *Response Times: The 3 Important Limits*. 1993. URL: <https://www.nngroup.com/articles/response-times-3-important-limits/>.
- [48] Zhiqing Sun et al. «MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices». En: *ACL*. 2020.